

NEXP-Completeness and Exponential Coefficient Growth for Existential Presburger Arithmetic with Divisibility

Ignacio Barros 

University of Antwerp, Belgium

Michaël Cadilhac 

DePaul University, USA

Guillermo A. Pérez 

University of Antwerp – Flanders Make, Belgium

Abstract

We prove that satisfiability for existential Presburger arithmetic with divisibility (EPAD) is **NEXP**-hard. Together with the known **NEXP** upper bound, this establishes **NEXP**-completeness. The lower bound is obtained by encoding succinct Boolean formulas whose satisfying assignments may have exponential length. A central difficulty is to represent, within a polynomial-size EPAD formula, the exponentially long integers arising in this encoding.

To address this difficulty, we introduce fixed-width arithmetic logic circuits (ALCs), whose gates perform addition, multiplication, and bit shifts. We show that polynomial-size uniform ALC families compute exactly the functions in **FPSPACE**, while their succinct exponential-size counterpart computes exactly the functions in **FEXP**. In both cases, the computed functions admit polynomial-size functional definitions in EPAD. This provides the compressed arithmetic needed for the lower-bound reduction.

The same construction yields **NEXP**-hardness for nonerasing word equations with Presburger length constraints over a fixed two-letter alphabet. We also study the elimination of divisibility constraints by enumerating their possible quotients. Earlier work identified an **NP** fragment in which the variables can be ordered so that divisibility dependencies always move forward through the order. We introduce a different fragment, called merge-absorptive, in which bounded-quotient divisibilities can successively connect all variable components. This fragment is polynomial-time recognizable and admits complete finite-quotient elimination, yet its satisfiability problem remains **NEXP**-complete. Finally, we show that the elimination process may necessarily produce equations with exponentially large coefficients.

2012 ACM Subject Classification Theory of computation → Logic and verification; Theory of computation → Algebraic complexity theory

Keywords and phrases Presburger arithmetic, divisibility, NEXP-completeness, succinct computation, arithmetic logic circuits, normalization

Funding This work was supported by the Belgian FWO-FNRS “SynthEx” (G0AH524N) project.

Acknowledgements We thank Alessio Mansutti for feedback on the normalization presentation and the connection to word equations with length constraints.

Contents

1	Introduction	3
2	Preliminaries and the Indexed-Bit Characterization of FPSPACE	4
3	Polynomial-Size ALCs Compute Exactly FPSPACE	6
3.1	From Polynomial-Size ALCs to FPSPACE	6

3.2	From FPSPACE to Polynomial-Size ALCs by Packed Truth Tables	7
4	Functional EPAD Definitions of Polynomial-Size ALCs	11
4.1	Number-Theoretic Bounds Used by the Power-of-Two Gadgets	12
4.2	EPAD Definitions of Binary-Indexed Powers of Two	12
4.3	Translation of ALC Gates into Functional EPAD	15
5	Encoding Succinct Exact-1-in-3 Satisfiability	16
5.1	Combining All Exact-1-in-3 Clauses into One Congruence	17
5.2	Encoding the Assignment Vector by One Integer	19
5.3	Expressing the Assignment Congruence by Square Divisibility	20
5.4	Polynomial-Space Indexed-Bit Computation for H and K	22
6	NEXP-Completeness of EPAD Satisfiability	24
7	Finite-Quotient Normalization and Exponential Coefficient Growth	25
7.1	A Merge-Absorptive Fragment with Complete Finite-Quotient Elimination . .	25
7.2	A Family Forcing Exponentially Large Primitive Coefficients	27
8	Summary of Results	28
A	Polynomial-Space Indexed-Bit Algorithms for H and K	30
B	Proof of the Merge-Absorptive Reduction	31
C	Proofs for Succinct ALCs	35
C.1	Functional EPAD Definitions of Succinct ALCs	36
D	NEXP-Hardness of Word Equations with Length Constraints	37

1 Introduction

Existential Presburger arithmetic is **NP**-complete [9]. Existential Presburger arithmetic with divisibility (EPAD) additionally allows atoms $L \mid M$, where L and M are affine terms. Such an atom holds when $M = qL$ for some integer q . Decidability was proved independently by Lipshitz [16] and Beltyukov [2]. Lechner, Ouaknine, and Worrell proved an **NEXP** upper bound [14]. Previously, only **NP**-hardness was known, and several works discussed the possibility of an **NP** upper bound [3, 8].

Divisibility constraints can enforce large fixed ratios without writing those ratios as coefficients. For example, polynomially many constraints can force $r = (2^{2^t} - 1)x$. We use this fact both in the hardness reduction and in the coefficient-growth example for finite-quotient elimination. The upper bound in the following theorem is known; our contribution is the matching lower bound.

► **Theorem 1** (Main theorem). *EPAD satisfiability is **NEXP**-complete under polynomial-time many-one reductions.*

Overview of the lower bound. We introduce fixed-width arithmetic logic circuits (ALCs) with modular addition, modular multiplication, and logical shifts. We write **ALC** for the functions computed by polynomial-time uniform polynomial-size ALC families and prove **ALC** = **FSPACE**. For the inclusion **FSPACE** $\subseteq$ **ALC**, a polynomial-space predicate for an indexed output bit is represented by a QBF, and packed truth tables evaluate all assignments simultaneously.

The function computed by each ALC gate is definable in EPAD. Polynomial-size homogeneous divisibility systems define multiplication by powers of two whose exponents are given in binary. A separate two-atom construction defines a bounded product. These formulas yield **ALC** $\subseteq$ **FEPAD**, and hence **FSPACE** $\subseteq$ **FEPAD**.

For a uniformly succinct Exact-1-in-3 formula, each clause condition is a linear equation in variables ranging over $\{-1, +1\}$. Powers of four combine all clause equations into one linear congruence, while an integer parameter specifies the values of the first assignment variables. A Chinese-remainder construction represents the $\{-1, +1\}$ -valued variables by one bounded integer, and two divisibility atoms involving differences of squares impose the congruence and the representation. A local conversion from the succinct Cook–Levin 3CNF to Exact-1-in-3 completes the reduction. The proof of Theorem 1 is given in Section 6.

Finite-quotient elimination. Suppose that the constraints on a branch imply that a quotient in $L \mid M$ belongs to a finite set $\mathcal{Q}$. The atom may then be replaced by one branch for each equation $M = cL$, with $c \in \mathcal{Q}$. We define a polynomial-time recognizable fragment, called merge-absorptive, in which bounded-quotient atoms can successively merge all variable components. This guarantees a sequence of replacements that eliminates every divisibility atom. Every EPAD formula has an equisatisfiable translation to this fragment, which proves **NEXP**-hardness. Since the fragment is contained in EPAD, the known upper bound applies, and its satisfiability problem is **NEXP**-complete.

Défossez, Haase, Mansutti, and Pérez identified an **NP** fragment based on a different restriction [5]. Its variables admit an order such that, among all nontrivial divisibilities entailed by the system, the greatest variable with respect to that order occurs only on the right-hand side. This prevents divisibility dependencies from feeding back toward earlier variables. The two conditions are incomparable: the ordered condition controls the direction of propagation, whereas merge-absorptivity makes quotient relations contractible.

We also give a polynomial-size family for which a specified replacement sequence produces primitive equations with coefficients of exponential bit length. For the same family, any primitive affine equation over the distinguished variables defining the projected relation has a coefficient of exponential bit length. The latter statement concerns a single projected equation, not arbitrary equivalent representations.

Contributions.

- We prove $\mathbf{ALC} = \mathbf{FPSPACE} \subseteq \mathbf{FEPAD}$ and, for the natural succinct variant $\mathbf{succALC}$ of $\mathbf{ALC}$, $\mathbf{succALC} = \mathbf{FEXP} \subseteq \mathbf{FEPAD}$.
- We give a square-divisibility encoding of succinct Exact-1-in-3 formulas in which an integer parameter specifies the values of an initial sequence of variables, and use it to prove $\mathbf{NEXP}$ -hardness of EPAD satisfiability.
- We introduce a polynomial-time recognizable fragment, called merge-absorptive, in which bounded-quotient divisibilities can successively connect all variable components. Unlike a known $\mathbf{NP}$ fragment based on a forward variable order, it remains $\mathbf{NEXP}$ -complete even though finite-quotient replacement eliminates all its divisibility atoms.
- We prove exponential coefficient growth along a specified replacement sequence and for any primitive affine equation defining the resulting projected relation.

A separate consequence of the lower-bound construction concerns word equations.

► **Corollary 2.** *Nonerasing satisfiability for positive Boolean combinations of word equations and quantifier-free Presburger length constraints over the fixed alphabet $\{a, b\}$ is $\mathbf{NEXP}$ -hard.*

Related works. Galota and Vollmer also characterized $\mathbf{FPSPACE}$ through arithmetic computations [6], but with a different size and uniformity convention. Their characterization describes an exponentially long arithmetic computation implicitly, through the ordered computation paths of a polynomial-time nondeterministic machine. It is therefore closer to a succinct description of an exponential-size circuit than to an ordinary polynomial-size uniform circuit family. Our characterization instead uses genuinely polynomial-size, polynomial-time uniform ALC families: the exponential amount of information is packed into fixed-width words rather than into an implicitly represented collection of gates. This form of the characterization is needed here, since it permits a gate-by-gate translation into a polynomial-size EPAD formula.

The linear congruence over $\{-1, +1\}$ -valued variables is related to the arithmetic encoding of Manders and Adleman [18], but here it is applied to a succinct formula and also fixes the values of a specified initial sequence of assignment variables. Succinct representations were introduced by Galperin and Wigderson [7]; the $\mathbf{NEXP}$ -complete succinct-satisfiability setting used here is due to Papadimitriou and Yannakakis [20]. Reductions from EPAD to word equations with length constraints also appear in [15, 4].

2 Preliminaries and the Indexed-Bit Characterization of $\mathbf{FPSPACE}$

Conventions. The expression $2^{\text{poly}(n)}$ denotes $2^{p(n)}$ for some polynomially bounded integer-valued function p . For $a > 0$, let $\text{len}(a) = \lfloor \log_2 a \rfloor + 1$, and put $\text{len}(0) = 1$. All written integers are binary encoded. All shifts are logical shifts of nonnegative binary words, and bit position zero is least significant.

An *affine-linear integer term* is $c + \sum_i a_i x_i$. It is *homogeneous* when $c = 0$.

► **Definition 3** (EPAD formula satisfiability). *An EPAD formula is an existential first-order formula over integer variables using Boolean combinations of affine-linear equalities, affine-linear inequalities, and divisibility atoms $L \mid M$ between affine-linear terms. EPAD satisfiability asks whether a given such formula is satisfiable.*

Divisibility has its ordinary integer meaning: $L \mid M$ if and only if $M = qL$ for some $q \in \mathbb{Z}$. Thus $0 \mid 0$ is true and $0 \mid M$ is false for $M \neq 0$. When we invoke a result stated under the convention that the divisor must be nonzero, we separate the case $L = M = 0$ by a Presburger disjunction, without further comment.

► **Example 4** (A small EPAD system). Take

$$0 < x < y < r,$$

together with the four atoms for the stage $h = 0$,

$$r + y \mid 4r, \quad y \mid r, \quad r - y \mid 2r, \quad r - 2y \mid r,$$

and the five atoms for the stage $h = 1$,

$$y + x \mid r + 3y, \quad x \mid y, \quad y - x \mid r + y, \quad y - 2x \mid r, \quad y - 3x \mid r - y.$$

This is the $t = 2$ instance of the systems in Lemma 19. The constraints force $y = 5x$ and $r = 15x$. Adding $v = r + x$ gives $v = 16x$. Divisibility can therefore describe large fixed multiples without writing them as coefficients. ◁

► **Definition 5** (Functional EPAD). *An n -input functional EPAD formula is an EPAD formula*

$$\Phi_n(\mathbf{x}, y) \equiv \exists \mathbf{z} \Theta_n(\mathbf{x}, y, \mathbf{z}),$$

where $\mathbf{x} \in \{0, 1\}^n$ are Boolean free inputs and $y \in \mathbb{N}$ is the distinguished output. It computes $f_n: \{0, 1\}^n \rightarrow \mathbb{N}$ if, for every $\mathbf{a} \in \{0, 1\}^n$, there is a unique $y \in \mathbb{N}$ satisfying $\Phi_n(\mathbf{a}, y)$, namely $f_n(\mathbf{a})$. The class **FEPAD** consists of functions computed by polynomial-time uniform polynomial-size families of such formulas. Zero-input families are allowed.

► **Definition 6** (Arithmetic logic circuits). *An arithmetic logic circuit (ALC) of word-width W is an acyclic circuit with a distinguished output gate whose values lie in $\{0, \dots, 2^W - 1\}$. It has Boolean inputs, constants 0, 1, and gates*

$$\begin{aligned} u +_W v &= (u + v) \bmod 2^W, & u \times_W v &= uv \bmod 2^W, \\ u \ll_W s &= 2^s u \bmod 2^W, & u \gg s &= \lfloor u / 2^s \rfloor. \end{aligned}$$

Shift offsets are nonnegative binary parameters in the circuit description; shifts by at least W return zero. A polynomial-size family is polynomial-time uniform if, from 1^n , a polynomial-time algorithm outputs the circuit, its word-width $W(n)$ in binary, where $1 \leq W(n) \leq 2^{\text{poly}(n)}$, and its output gate. The output word is a nonnegative binary integer. The resulting function class is **ALC**.

► **Definition 7** (Function classes). *The class **FPSpace** consists of functions from Boolean strings to nonnegative integers computed by deterministic polynomial-space transducers with a one-way write-only output tape. The class **FEXP** is defined in the same way using deterministic $2^{\text{poly}(n)}$ -time transducers. Outputs are read in binary, and leading zeros are immaterial. The class **NEXP** consists of languages accepted by nondeterministic machines in time $2^{\text{poly}(n)}$.*

► **Lemma 8** (Indexed-bit characterization). *A function family $f_n: \{0,1\}^n \rightarrow \mathbb{N}$ belongs to **FPSPACE** if and only if there are a polynomial p and a **PSPACE** predicate $\mathcal{B}$ such that*

$$f_n(\mathbf{x}) < 2^{2^{p(n)}}$$

and, for every binary index $0 \leq i < 2^{p(n)}$, $\mathcal{B}(\mathbf{x}, i) = 1$ exactly when the i th binary bit of $f_n(\mathbf{x})$ is 1.

Proof. Suppose first that a polynomial-space machine halts after writing the output. Its state, input-head positions, and work tapes have only exponentially many possible configurations. The output tape is write-only, so a repeated configuration would force an infinite computation. The machine therefore writes at most exponentially many bits. To compute one indexed output bit, simulate the machine once to determine the output length and a second time to return the symbol at the requested least-significant-bit index. Both simulations use a polynomial-bit counter.

Conversely, a polynomial-space transducer scans the exponentially many indices from the largest down to zero, evaluates $\mathcal{B}$ at each index, and writes the returned bit. The index counter has polynomial length, and the output tape is not charged to the space bound. ◀

This characterization is standard; see, for example, [12].

3 Polynomial-Size ALCs Compute Exactly FPSPACE

To prove $\mathbf{ALC} \subseteq \mathbf{FPSPACE}$, we evaluate a requested output bit using polynomial space. For the converse inclusion, we convert a polynomial-space indexed-bit predicate to a QBF and represent its truth table in a large integer. Arithmetic operations on this representation evaluate all index assignments simultaneously.

3.1 From Polynomial-Size ALCs to FPSPACE

► **Proposition 9.** *Every polynomial-time uniform polynomial-size ALC family computes a function in **FPSPACE**.*

Proof. Fix a circuit, an input, and a requested output-bit position $i < W$. Evaluate gate bits recursively. Input and constant bits are read directly. A left shift replaces index i by $i - s$, returning zero if $i < s$; a right shift replaces it by $i + s$, returning zero if $i + s \geq W$.

For addition modulo 2^W , scan columns zero through i and maintain the one-bit carry. Higher columns cannot affect bit i . For multiplication modulo 2^W , at column t compute

$$w_t = \sum_{j=0}^t u_j v_{t-j},$$

where u_j and v_j are recursively obtained predecessor bits, and maintain $c_0 = 0$ and $c_{t+1} = \lfloor (c_t + w_t)/2 \rfloor$. Since $w_t \leq t + 1$, induction gives $c_t \leq t$, so the carry has $O(\log(t + 1))$ bits. The requested product bit is $(c_i + w_i) \bmod 2$.

Gate indices, bit indices, and carries have polynomial length, and the recursion stack has depth at most the number of gates. Each output bit is therefore computable in polynomial space. The machine writes the W output bits from most significant to least significant, using the binary representation of W as its counter bound. ◀

3.2 From FPSPACE to Polynomial-Size ALCs by Packed Truth Tables

A packed truth table places the values of a Boolean function at widely separated bit positions. The proof needs to retain, from two packed tables, the products of entries belonging to the same assignment. We use this operation for pointwise conjunction, restriction of a variable, and the final placement of the output bits.

For $\mathbf{a} = (a_0, \dots, a_{k-1}) \in \{0, 1\}^k$, let $\text{enc}(\mathbf{a})$ be the integer whose base-five digits are $a_0, \dots, a_{k-1}$, from least to most significant:

$$\text{enc}(\mathbf{a}) = \sum_{i=0}^{k-1} a_i 5^i.$$

For a Boolean function $F: \{0, 1\}^k \rightarrow \{0, 1\}$ and a positive integer R , define its packed truth table at scale R by

$$\langle F \rangle_R = \sum_{\mathbf{a} \in \{0, 1\}^k} F(\mathbf{a}) 2^{R \text{enc}(\mathbf{a})}.$$

Thus the bit at position $R \text{enc}(\mathbf{a})$ is $F(\mathbf{a})$.

Let $\boldsymbol{\delta} = (\delta_0, \dots, \delta_{k-1})$ be a tuple of nonnegative integers. Define

$$\mathcal{M}_{R, \boldsymbol{\delta}}(F, G) = \langle F \rangle_R \langle G \rangle_R \prod_{i=0}^{k-1} \left(2^{2R5^i} + 2^{\delta_i} \right). \quad (1)$$

This product contains terms arising from every pair of assignments to F and G . The next lemma identifies the consecutive bits that retain exactly the pairs in which the two assignments are equal. For an integer x , write

$$\text{block}_{h,d}(x) = \left\lfloor \frac{x}{2^h} \right\rfloor \bmod 2^d$$

for the d consecutive bits of x beginning at position h .

► **Lemma 10** (Diagonal extraction). *Let $F, G: \{0, 1\}^k \rightarrow \{0, 1\}$, let $\boldsymbol{\delta} \in \mathbb{N}^k$, and put*

$$h = R \frac{5^k - 1}{2}.$$

If

$$R > \sum_{i=0}^{k-1} \delta_i + 3k + 1,$$

then

$$\text{block}_{h,R}(\mathcal{M}_{R, \boldsymbol{\delta}}(F, G)) = \sum_{\mathbf{a} \in \{0, 1\}^k} F(\mathbf{a}) G(\mathbf{a}) 2^{\sum_{i=0}^{k-1} a_i \delta_i}.$$

Proof. Expand $\mathcal{M}_{R, \boldsymbol{\delta}}(F, G)$. A term is determined by $\mathbf{a}, \mathbf{b}, \mathbf{z} \in \{0, 1\}^k$, where $\mathbf{a}$ and $\mathbf{b}$ select entries from the two packed tables and z_i records the choice of 2^{2R5^i} from the i -th factor in (1). Its exponent is $Rq + r$, where

$$q = \sum_{i=0}^{k-1} (a_i + b_i + 2z_i) 5^i, \quad r = \sum_{i=0}^{k-1} (1 - z_i) \delta_i.$$

Every base-five digit of q lies in $\{0, 1, 2, 3, 4\}$, so these digits produce no carries. Since

$$\frac{5^k - 1}{2} = \sum_{i=0}^{k-1} 2 \cdot 5^i,$$

we have $Rq = h$ exactly when $a_i = b_i$ and $z_i = 1 - a_i$ for every i . For those terms, $r = \sum_i a_i \delta_i$, and their sum is the right-hand side of the claimed identity.

It remains to check that no other term contributes to the block beginning at h . If $Rq < h$, then the exponent is at most $h - R + \sum_i \delta_i$. There are at most 2^{3k} terms, and the assumed inequality prevents their sum from carrying into position h . If $Rq > h$, then the exponent is at least $h + R$, beyond the extracted block. Finally, the retained sum is at most $2^{k+\sum_i \delta_i} < 2^R$, so it fits within that block. $\blacktriangleleft$

► **Corollary 11** (Pointwise product). *Let $F, G: \{0, 1\}^k \rightarrow \{0, 1\}$, set $\delta_i = R'5^i$, and put $h = R(5^k - 1)/2$. If*

$$R > R' \frac{5^k - 1}{4} + 3k + 1,$$

then

$$\text{block}_{h,R}(\mathcal{M}_{R,\delta}(F, G)) = \langle F \wedge G \rangle_{R'}.$$

Proof. In Lemma 10, the exponent attached to $\mathbf{a}$ is $R' \text{enc}(\mathbf{a})$, and $F(\mathbf{a})G(\mathbf{a}) = (F \wedge G)(\mathbf{a})$. $\blacktriangleleft$

► **Corollary 12** (Restriction). *Let $F: \{0, 1\}^{k+1} \rightarrow \{0, 1\}$, write its last argument as y , and fix $b \in \{0, 1\}$. Define*

$$F_b(\mathbf{a}) = F(\mathbf{a}, b), \quad \chi_b(\mathbf{a}, y) = \begin{cases} 1, & y = b, \\ 0, & y \neq b. \end{cases}$$

Set $\delta_i = R'5^i$ for $i < k$, set $\delta_k = 0$, and put $h = R(5^{k+1} - 1)/2$. If

$$R > R' \frac{5^k - 1}{4} + 3(k + 1) + 1,$$

then

$$\text{block}_{h,R}(\mathcal{M}_{R,\delta}(F, \chi_b)) = \langle F_b \rangle_{R'}.$$

Proof. The packed table used in $\mathcal{M}_{R,\delta}(F, \chi_b)$ is

$$\langle \chi_b \rangle_R = \prod_{i=0}^{k-1} (1 + 2^{R5^i}) \begin{cases} 1, & b = 0, \\ 2^{R5^k}, & b = 1. \end{cases}$$

Apply Lemma 10. The factor $\chi_b(\mathbf{a}, y)$ removes the terms with $y \neq b$, and $\delta_k = 0$ removes the fixed coordinate from the output positions. $\blacktriangleleft$

► **Theorem 13.** *Every function in **FPSpace** is computed by a polynomial-time uniform family of polynomial-size ALCs.*

Proof. We construct an ALC of word-width W , to be fixed at the end of the proof. The value of W will exceed the bit length of every integer used below, so fixed-width addition and multiplication agree with ordinary integer addition and multiplication. The circuit forms 2^E as $1 \ll_W E$. For $h + d \leq W$, it extracts the d -bit block of an intermediate value x beginning at position h by

$$((x \gg h) \ll_W (W - d)) \gg (W - d). \quad (2)$$

Let $f_n: \{0, 1\}^n \rightarrow \mathbb{N}$ be in **FPSPACE**. By Lemma 8, fix a polynomial p and the corresponding **PSPACE** indexed-bit predicate. Using the $p(n)$ -bit binary representation of the index, with the least significant bit first, write this predicate as $\mathcal{B}(\mathbf{x}, \mathbf{a})$, where $\mathbf{a} \in \{0, 1\}^{p(n)}$.

The polynomial-space-to-QBF reduction [19, Chapter 19] gives, uniformly in n , a polynomial-size prenex quantified Boolean formula with free variables $\mathbf{x}, \mathbf{a}$:

$$\Psi_n(\mathbf{x}, \mathbf{a}) = Q_0 y_0 \cdots Q_{m-1} y_{m-1} \varphi_n(\mathbf{x}, \mathbf{a}, \mathbf{y}).$$

It is true exactly when $\mathcal{B}(\mathbf{x}, \mathbf{a}) = 1$. We pack the variables in the order $\mathbf{a}, y_0, \dots, y_{m-1}$; the variables $\mathbf{x}$ remain ALC inputs. We eliminate $y_{m-1}, \dots, y_0$ in this order, so the variable removed at each step is the last variable indexing the current table.

For a subformula θ and $b \in \{0, 1\}$, let F_θ^b be the Boolean function of the variables indexing the current table, with $\mathbf{x}$ treated as parameters, that equals 1 exactly when θ has truth value b . The construction produces $\langle F_\theta^0 \rangle_R$ and $\langle F_\theta^1 \rangle_R$ at any requested output scale R . At a connective or quantifier, it chooses one common larger scale for the operand tables, large enough to satisfy the relevant inequality in Corollaries 11 and 12; taking the right-hand side plus one suffices. Hence the operand tables used at one step all have the same scale.

Literals. Suppose the current table is indexed by $v_0, \dots, v_{k-1}$. Its all-ones table at scale R is

$$A_{k,R} = \sum_{\mathbf{a} \in \{0,1\}^k} 2^{R \text{enc}(\mathbf{a})} = \prod_{i=0}^{k-1} (1 + 2^{R5^i}).$$

For the literal v_j ,

$$\langle F_{v_j}^1 \rangle_R = 2^{R5^j} \prod_{i \neq j} (1 + 2^{R5^i}), \quad \langle F_{v_j}^0 \rangle_R = \prod_{i \neq j} (1 + 2^{R5^i}).$$

For an ALC input bit x_t , put

$$\bar{x}_t = ((x_t +_W 1) \ll_W (W - 1)) \gg (W - 1).$$

Since $x_t \in \{0, 1\}$, this value is $1 - x_t$. Thus

$$\langle F_{x_t}^1 \rangle_R = x_t A_{k,R}, \quad \langle F_{x_t}^0 \rangle_R = \bar{x}_t A_{k,R}.$$

Boolean connectives. The characteristic functions satisfy

$$\begin{aligned} F_{\alpha \wedge \beta}^1 &= F_\alpha^1 F_\beta^1, & F_{\alpha \wedge \beta}^0 &= F_\alpha^0 F_\beta^0 + F_\alpha^0 F_\beta^1 + F_\alpha^1 F_\beta^0, \\ F_{\alpha \vee \beta}^0 &= F_\alpha^0 F_\beta^0, & F_{\alpha \vee \beta}^1 &= F_\alpha^0 F_\beta^1 + F_\alpha^1 F_\beta^0 + F_\alpha^1 F_\beta^1. \end{aligned}$$

Products here are pointwise products of Boolean functions. Each is implemented by Corollary 11 at the requested output scale. For each assignment, at most one summand in either

three-term sum is 1. Adding the corresponding packed tables therefore produces no carry and gives the packed table of the sum. Negation satisfies $F_{\neg\alpha}^b = F_{\alpha}^{1-b}$.

Quantifiers. Suppose y is the last variable indexing the table for θ . Corollary 12 gives the packed tables of $\mathbf{a} \mapsto F_{\theta}^c(\mathbf{a}, b)$ for $b, c \in \{0, 1\}$. The required characteristic functions are

$$\begin{aligned} F_{\exists y \theta}^0(\mathbf{a}) &= F_{\theta}^0(\mathbf{a}, 0)F_{\theta}^0(\mathbf{a}, 1), \\ F_{\exists y \theta}^1(\mathbf{a}) &= F_{\theta}^0(\mathbf{a}, 0)F_{\theta}^1(\mathbf{a}, 1) + F_{\theta}^1(\mathbf{a}, 0)F_{\theta}^0(\mathbf{a}, 1) + F_{\theta}^1(\mathbf{a}, 0)F_{\theta}^1(\mathbf{a}, 1), \\ F_{\forall y \theta}^1(\mathbf{a}) &= F_{\theta}^1(\mathbf{a}, 0)F_{\theta}^1(\mathbf{a}, 1), \\ F_{\forall y \theta}^0(\mathbf{a}) &= F_{\theta}^0(\mathbf{a}, 0)F_{\theta}^0(\mathbf{a}, 1) + F_{\theta}^0(\mathbf{a}, 0)F_{\theta}^1(\mathbf{a}, 1) + F_{\theta}^1(\mathbf{a}, 0)F_{\theta}^0(\mathbf{a}, 1). \end{aligned}$$

As in the connective case, Corollary 11 implements each product. For each assignment, at most one summand in a three-term sum is 1, so adding the packed tables produces no carry.

If a table indexed by k variables is requested at scale R' , either inequality in Corollaries 11 and 12 can be met with a scale R satisfying $\log R \leq \log R' + O(k)$. The QBF has polynomial size, so the recursion has polynomial depth and $k = \text{poly}(n)$ at every step. Consequently every scale used by the construction has a polynomial-length binary representation and is computable in polynomial time.

Output. Choose the requested scale of the final table so that

$$R > 2^{p(n)} + 3p(n).$$

After eliminating all quantified variables, the construction gives

$$P_R(\mathbf{x}) = \langle F_{\Psi_n}^1 \rangle_R = \sum_{\mathbf{a} \in \{0,1\}^{p(n)}} \mathcal{B}(\mathbf{x}, \mathbf{a}) 2^{R \text{enc}(\mathbf{a})}.$$

Set $\delta_j = 2^j$ and

$$h = R \frac{5^{p(n)} - 1}{2}.$$

By Lemma 10,

$$\text{block}_{h,R}(\mathcal{M}_{R,\delta}(F_{\Psi_n}^1, F_{\Psi_n}^1)) = \sum_{\mathbf{a} \in \{0,1\}^{p(n)}} \mathcal{B}(\mathbf{x}, \mathbf{a}) 2^{\sum_j a_j 2^j} = f_n(\mathbf{x}).$$

The output has at most $2^{p(n)}$ bits by Lemma 8. The circuit therefore extracts the $2^{p(n)}$ -bit block beginning at h using (2).

Each subformula occurrence is constructed once at its requested scale, and forming $\mathcal{M}_{R,\delta}(F, G)$ uses $O(k)$ gates. Since the QBF has polynomial size, the resulting ALC has polynomial size.

It remains to choose W . All constants, exponents, scales, and block endpoints above have polynomial-length binary representations. Along the polynomial-size acyclic calculation, addition increases bit length by at most one, multiplication adds the bit lengths of its inputs, and a left shift by E increases bit length by at most E . Therefore a value $W \leq 2^{\text{poly}(n)}$, with a polynomial-length binary representation, can be chosen larger than the bit length of every intermediate integer and larger than every block endpoint. With this word-width, all preceding calculations are valid ALC operations. The QBF reduction, the recursive scale choices, and the gate construction are polynomial-time uniform. $\blacktriangleleft$

► Theorem 14. *Polynomial-time uniform polynomial-size ALC families compute exactly FSPACE: ALC = FSPACE.*

Proof. Combine Proposition 9 and Theorem 13. ◀

► **Remark 15 (Succinct ALCs).** A succinct ALC family has word-width $W(n) \leq 2^{\text{poly}(n)}$ and gates indexed by strings of polynomial length. Every gate index specifies, through a polynomial-time direct-connection algorithm, the gate type, its predecessor indices, its shift offset when applicable, and whether it is the output gate; for an input gate, the algorithm also specifies the indexed Boolean input bit. Predecessor indices are smaller than the gate index, and unused indices may denote zero gates. The word-width and output index are polynomial-time computable. The functions computed by these families form the class **succALC**. As proved in Proposition 51,

$$\text{succALC} = \text{FEXP}.$$

Unlike **FSPACE**, the class **FEXP** permits outputs of exponential length: on inputs of length n , an output may contain $2^{\text{poly}(n)}$ bits and thus represent an integer as large as $2^{2^{\text{poly}(n)}} - 1$. This is why the model allows exponential word-width.

The next proposition shows that adding the listed fixed-width arithmetic and logic operations does not change the function class defined in Definition 6. It justifies the name of the model but is not used in the sequel.

► **Proposition 16 (Completeness for standard fixed-width operations).** *Let ALC^{std} be defined like **ALC**, but with the following additional gate types: fixed-width subtraction, two's-complement negation, unsigned quotient and remainder; bitwise negation, conjunction, disjunction, and exclusive disjunction; unsigned equality and order comparisons; and logical shifts and left or right rotations whose offset may be supplied by another operand. Arithmetic results are reduced modulo 2^W , comparisons return 0 or 1, shifts by at least W return zero, and rotations use the offset modulo W . Under any fixed convention for division by zero,*

$$\text{ALC}^{\text{std}} = \text{ALC}.$$

Proof. Every circuit over the original basis is also a circuit over the extended basis. For the converse, consider a polynomial-time uniform polynomial-size family over the extended basis. Standard locally uniform Boolean circuits of size $W^{O(1)}$ and depth $(\log W)^{O(1)}$ implement every additional gate; for division and remainder, use the uniform threshold circuits of Hesse, Allender, and Barrington [10, 11]. Recursively evaluating one requested output bit stores only polynomially many gate indices and counters, each of length $O(\log W)$. The enclosing ALC has polynomially many gates and $\log W = \text{poly}(n)$, so its indexed-output-bit predicate is in **PSPACE**. The output has at most W bits; hence Lemma 8 and Theorem 14 gives an equivalent polynomial-time uniform polynomial-size ALC family over the original basis. ◀

4 Functional EPAD Definitions of Polynomial-Size ALCs

We translate each gate of a polynomial-size ALC into EPAD. Addition and right shift use quotient–remainder equations. The other constructions needed for the translation define multiplication by a power of two whose exponent is given in binary and define the bounded product of two variables. The resulting translation proves $\text{ALC} \subseteq \text{FEPAD}$.

If a formula has free variables x, r , its (x, r) -*projection* is the set of pairs that extend to values of its auxiliary existential variables satisfying the formula. The power-of-two constructions in this section use linear equalities, divisibility atoms, and positivity and order constraints.

A *homogeneous divisibility system* consists of divisibility atoms between homogeneous integer linear forms and homogeneous linear side conditions. It is *positive* when all variables

range over positive integers. This terminology is used again for the merge-absorptive fragment in Section 7.1.

4.1 Number-Theoretic Bounds Used by the Power-of-Two Gadgets

► **Lemma 17** (Mersenne divisibility). *For positive integers u, v ,*

$$2^u - 1 \mid 2^v - 1 \iff u \mid v.$$

Proof. If $v = ku$, factor $(2^u)^k - 1$. Conversely, write $v = qu + r$ with $0 \leq r < u$. Modulo $2^u - 1$, one has $2^v - 1 \equiv 2^r - 1$. Divisibility then forces $r = 0$, since $0 \leq 2^r - 1 < 2^u - 1$. ◀

► **Lemma 18** (LCM of a short interval). *The least common multiple of any $d + 1$ consecutive positive integers is at least*

$$\frac{2^{d+1}}{d+2}.$$

Proof. Put $n = d + 1$. Among n consecutive integers, every residue class modulo a prime power $p^e \leq n$ occurs at least once; in particular the interval contains a multiple of p^e . Its lcm is therefore divisible by $\text{lcm}(1, \dots, n)$. For each $0 \leq k \leq n$, the binomial coefficient $\binom{n}{k}$ divides this lcm. Indeed, by Legendre's formula, the exponent of a prime p in $\binom{n}{k}$ is

$$\sum_{e \geq 1} \left(\left\lfloor \frac{n}{p^e} \right\rfloor - \left\lfloor \frac{k}{p^e} \right\rfloor - \left\lfloor \frac{n-k}{p^e} \right\rfloor \right).$$

Each summand is zero or one, and there are at most $\lfloor \log_p n \rfloor$ nonzero terms. Since the $n + 1$ binomial coefficients sum to 2^n , one is at least $2^n / (n + 1)$. ◀

4.2 EPAD Definitions of Binary-Indexed Powers of Two

The next construction defines multiplication by $2^{2^t} - 1$ using polynomially many constraints. Its variables form a chain whose successive ratios are forced to be the Fermat numbers $2^{2^h} + 1$. Some atoms restrict a ratio to a Fermat number, and additional atoms exclude the incorrect choices.

► **Lemma 19** (Mersenne multiplier). *For every integer $t \geq 0$, there is a polynomial-size positive homogeneous divisibility system $\Delta_t(x, r)$ whose (x, r) -projection is*

$$\{(x, (2^{2^t} - 1)x) : x \in \mathbb{N}_{>0}\}.$$

Adding the equality $v = r + x$ gives projection

$$\{(x, 2^{2^t}x) : x \in \mathbb{N}_{>0}\}.$$

Proof. Use variables

$$0 < y_0 < y_1 < \dots < y_t, \quad y_0 = x, \quad y_t = r.$$

When $t = 0$, these conditions reduce to $r = x$. For $i = t, t - 1, \dots, 1$, put $h = t - i$ and include

$$y_i + y_{i-1} \mid y_t + 3y_i, \tag{3}$$

$$y_{i-1} \mid y_i, \tag{4}$$

$$y_i - y_{i-1} \mid y_t + y_i, \tag{5}$$

$$y_i - 2y_{i-1} \mid y_t. \tag{6}$$

For $h \geq 1$, let m_h be the least nonnegative integer satisfying

$$\frac{2^{m_h+1}}{m_h+2} > h,$$

put $D_h = \min(m_h, h-1)$, and, for $0 \leq q \leq D_h$, include

$$y_i - (2^{2^q} + 1)y_{i-1} \mid y_t + (1 - 2^{2^q})y_i. \quad (7)$$

Atom (3) is not needed for the projection asserted in this lemma. It is included because it is a bounded-divisor merge in Lemma 40, and its finite-quotient replacement yields the equation used in Lemma 45.

Soundness. We prove, for $i = t, t-1, \dots, 1$, that

$$y_t = (B-1)y_i, \quad B = 2^{2^{t-i}},$$

implies $y_i = (B+1)y_{i-1}$. The hypothesis is an identity when $i = t$. By (4), write $y_i = Qy_{i-1}$ with $Q \geq 2$. Equation (5) gives $Q-1 \mid BQ$, and $\gcd(Q-1, Q) = 1$, so $Q-1 \mid B$. Thus $Q = 2^s + 1$ for some $s \geq 0$. The case $s = 0$ contradicts (6); hence $s > 0$. From (6) and $\gcd(2^s - 1, 2^s + 1) = 1$,

$$2^s - 1 \mid B - 1 = 2^{2^{t-i}} - 1.$$

By Lemma 17, $s \mid 2^{t-i}$, so $s = 2^a$ for some $0 \leq a \leq h = t - i$, and $Q = 2^{2^a} + 1$. If $h = 0$, then $a = h$.

Assume $h > 0$ and $a < h$. If $a \leq D_h$, the exclusion atom with $q = a$ has zero left-hand side and positive right-hand side, a contradiction. Thus $a > D_h$. Fix $0 \leq q \leq D_h$. The coefficient $2^{2^a} - 2^{2^q}$ is coprime to $2^{2^a} + 1$. Indeed, if an odd prime p divided both, then $2^{2^a} \equiv -1 \pmod{p}$, so the multiplicative order of 2 modulo p would be 2^{a+1} ; the congruence $2^{2^a} \equiv 2^{2^q} \pmod{p}$ would instead make that order divide $2^a - 2^q = 2^q(2^{a-q} - 1)$, which is impossible. The exclusion atom therefore implies

$$2^{2^a} - 2^{2^q} \mid 2^{2^h} - 2^{2^q}.$$

Cancelling the common factor 2^{2^q} gives

$$2^{2^a-2^q} - 1 \mid 2^{2^h-2^q} - 1.$$

By Lemma 17,

$$2^a - 2^q \mid 2^h - 2^q.$$

After cancelling 2^q , another application of Lemma 17 gives

$$a - q \mid h - q, \quad \text{and hence} \quad a - q \mid h - a.$$

If $D_h = h - 1$, then $a > D_h$ contradicts $a < h$. Otherwise $D_h = m_h$, and every integer in $a - m_h, \dots, a$ divides the positive integer $h - a < h$. By Lemma 18, their least common multiple exceeds h , a contradiction. Thus $a = h$, and

$$y_i = (B+1)y_{i-1}, \quad y_t = (B^2 - 1)y_{i-1}.$$

After stage $i = 1$, this gives $y_t = (2^{2^t} - 1)y_0$.

Completeness. Conversely, fix $x = y_0 > 0$ and set

$$y_i = (2^{2^{t-i}} + 1)y_{i-1} \quad (1 \leq i \leq t).$$

The identity

$$\prod_{q=0}^{h-1} (2^{2^q} + 1) = 2^{2^h} - 1$$

shows that, at stage i , with $h = t - i$ and $B = 2^{2^h}$,

$$y_t = (B - 1)y_i, \quad y_i = (B + 1)y_{i-1}.$$

The quotient in (3) is then

$$\frac{y_t + 3y_i}{y_i + y_{i-1}} = B + 1.$$

The quotients in (4)–(6) are also $B + 1$. In an exclusion atom, both sides contain the factor $2^{2^h} - 2^{2^q}$, and the quotient is $2^{2^h} + 1$. Thus every positive x extends to a solution.

Size. The formula has polynomial size. For $h \geq 1$, the choice $m = \lceil 2 \log_2(h + 2) \rceil + 2$ satisfies $2^{m+1}/(m + 2) > h$, so $m_h = O(\log h)$. There are $O(t \log t)$ exclusion atoms, and the largest coefficient 2^{2^q} has polynomial binary length. $\blacktriangleleft$

► **Remark 20.** At stage h , explicitly excluding every incorrect Fermat ratio would require coefficients 2^{2^q} for values of q close to h . Those coefficients have exponential binary length in h , even though there are only h possible ratios. The construction writes exclusions only for $q = O(\log h)$, where the coefficients have polynomial binary length. The interval-lcm bound in Lemma 18 then rules out all remaining smaller ratios simultaneously.

► **Lemma 21** (Succinct positive power scaling). *Given $e \geq 0$ in binary, there is a polynomial-size positive homogeneous divisibility system whose (u, v) -projection is*

$$\{(u, 2^e u) : u \in \mathbb{N}_{>0}\}.$$

Proof. For $e = 0$, use $v = u$. Assume $e > 0$ and let $J = \{j : e_j = 1\}$ for the binary expansion $e = \sum_{j \in J} 2^j$. Enumerate J and introduce a chain $u = z_0, z_1, \dots, z_m = v$. At the step corresponding to j , use Lemma 19 to force $r = (2^{2^j} - 1)z_\ell$ and add $z_{\ell+1} = r + z_\ell$. The chain forces $v = 2^e u$. There are at most $\text{len}(e)$ steps, each of size polynomial in $\text{len}(e)$. $\blacktriangleleft$

► **Lemma 22** (Signed power-of-two scaling). *Given $e \geq 0$ in binary, there is a polynomial-size EPAD formula $\text{Scale}_e(A, B)$ over free integer variables A, B such that*

$$\text{Scale}_e(A, B) \iff B = 2^e A.$$

Proof. Introduce positive variables A^+, A^-, B^+, B^- , impose $A = A^+ - A^-$ and $B = B^+ - B^-$, and use two copies of Lemma 21 to force $B^+ = 2^e A^+$ and $B^- = 2^e A^-$. Every integer is a difference of two positive integers, so the construction is complete as well as sound. $\blacktriangleleft$

The next lemma defines a bounded product using two divisibility atoms. The first gives $W = qV$ for an integer q . The second implies $q = U$ because the bounds make $|U - q|$ smaller than $|1 + \Lambda V|$.

► **Lemma 23** (Quotient-sharing product). *Let U, V, W be integer affine forms satisfying*

$$|U| < M_U, \quad |W| < M_W,$$

and let $\Lambda = 2^h > M_U + M_W + 1$. Multiples by Λ are represented by fresh variables constrained with Lemma 22. Under these bounds, the atoms

$$V \mid W, \quad 1 + \Lambda V \mid U + \Lambda W \tag{8}$$

are equivalent to $W = UV$.

Proof. If $W = UV$, both atoms hold. Conversely, if $V = 0$, then $V \mid W$ forces $W = 0$. If $V \neq 0$, write $W = qV$. The bound on W gives $|q| < M_W$. Modulo $1 + \Lambda V$, the second atom gives $1 + \Lambda V \mid U - q$. But

$$|U - q| < M_U + M_W < \Lambda - 1 \leq |1 + \Lambda V|,$$

so $U = q$ and $W = UV$. ◀

4.3 Translation of ALC Gates into Functional EPAD

The translation introduces one bounded integer variable for each circuit gate. Once the predecessor values are fixed, the constraints for a gate determine its value uniquely.

► **Theorem 24.** *For every ALC of word-width W , one can construct an EPAD formula in time polynomial in the circuit description and in $\text{len}(W)$. The formula defines the graph of the function computed by the circuit and is functional in the distinguished output. Consequently,*

$$\text{ALC} \subseteq \text{FEPAD}.$$

Proof. Let C be an ALC of word-width W . Generate variables $P = 2^W$ and $P_2 = 2^{2W}$ from the constant one by Lemma 22. For every gate g , introduce a value variable Y_g and impose $0 \leq Y_g < P$. Boolean input gates are identified with their free input variables and constrained by $0 \leq x_i \leq 1$; constant gates are fixed to zero or one.

Addition. For an addition gate $g = u +_W v$, impose the disjunction

$$Y_u + Y_v = Y_g \quad \vee \quad Y_u + Y_v = Y_g + P.$$

Since $0 \leq Y_u + Y_v < 2P$, this forces the unique remainder modulo P .

Multiplication. For a multiplication gate $g = u \times_W v$, quantify T with $0 \leq T < P_2$. Apply Lemma 23 to force $T = Y_u Y_v$, using operand bound 2^W , product bound 2^{2W} , and $\Lambda = 2^{2W+2}$. Then impose $P \mid T - Y_g$. The bound $0 \leq Y_g < P$ makes Y_g the unique remainder of T modulo P .

Left shift. For a left shift $g = u \ll_W s$, if $s \geq W$, impose $Y_g = 0$. Otherwise, quantify T , use $\text{Scale}_s(Y_u, T)$, and impose $P \mid T - Y_g$. Since $s < W$ and $Y_u < P$, one has $T < P_2$.

Right shift. For a right shift $g = u \gg s$, if $s \geq W$, impose $Y_g = 0$. Otherwise, generate $P_s = 2^s$, quantify R, Q , impose

$$\text{Scale}_s(Y_g, Q), \quad Y_u = R + Q, \quad 0 \leq R < P_s.$$

This is the Euclidean decomposition of Y_u by 2^s , and therefore forces $Y_g = \lfloor Y_u / 2^s \rfloor$.

Identify the formula output y with the value of the distinguished circuit gate. Induction over the circuit shows that every gate value is uniquely determined. Each gate contributes polynomially many constraints, and every exponent has polynomial binary length in the circuit description. ◀

► **Corollary 25.** *One has $\text{FPSPACE} \subseteq \text{FEPAD}$.*

Proof. Combine Theorems 14 and 24. ◀

► **Remark 26.** For the succinct model of Remark 15, the appendix gives polynomial-size functional EPAD definitions of the graphs of the functions computed by these circuits; see Theorem 54. Together with $\text{succALC} = \text{FEXP}$, this yields

$$\text{FEXP} \subseteq \text{FEPAD}.$$

5 Encoding Succinct Exact-1-in-3 Satisfiability by Two Square-Divisibility Conditions

We prove that, for a succinct Exact-1-in-3 formula F_n , the existence of a satisfying assignment whose first L truth values encode an integer Z can be expressed using one nonnegative integer witness and two divisibility conditions of the form $a \mid b^2 - c^2$. We first state the result and then prove it in the four subsections below.

► **Definition 27** (Uniformly succinct Exact-1-in-3 family). *Let $m = m(n)$ and $\ell = \ell(n)$ be positive integers bounded by $2^{\text{poly}(n)}$, with binary representations computable in polynomial time. An Exact-1-in-3 clause is an ordered triple of literals and is satisfied when exactly one of its three literal occurrences is true. Repeated literals are allowed and are counted with multiplicity.*

A clause oracle, on input $(1^n, k)$ with $0 \leq k < m$ given in binary, returns three pairs (i_h, ε_h) , for $h \in \{1, 2, 3\}$, where $0 \leq i_h < \ell$ is given in binary and $\varepsilon_h \in \{-1, +1\}$. The pair (i_h, ε_h) denotes the literal u_{i_h} when $\varepsilon_h = +1$, and $\neg u_{i_h}$ when $\varepsilon_h = -1$. The three returned pairs form clause k of an Exact-1-in-3 formula F_n over variables $u_0, \dots, u_{\ell-1}$.

Let $L = L(n)$ be polynomial-time computable and satisfy $0 \leq L \leq \ell$. For an assignment $\mathbf{u} \in \{0, 1\}^\ell$, define

$$\text{word}_L(\mathbf{u}) = \sum_{b=0}^{L-1} u_b 2^b.$$

Thus $\text{word}_L(\mathbf{u}) = Z$ means that $u_0, \dots, u_{L-1}$ are the binary digits of Z , from least to most significant. We call $(F_n, L)_n$ uniformly succinct if the clause oracle runs in time polynomial in n .

► **Theorem 28** (Succinct Exact-1-in-3 normal form). *For every uniformly succinct family $(F_n, L)_n$ as in Definition 27, one can define a power of two M , positive integers H, K , and an affine function $\rho(Z)$ that is positive for $0 \leq Z < 2^L$, such that the following holds. Given 1^n and a binary bit position, the corresponding bit of H or K is computable in polynomial space. For every integer Z with $0 \leq Z < 2^L$,*

$$F_n \text{ has a satisfying assignment } \mathbf{u} \text{ with } \text{word}_L(\mathbf{u}) = Z$$

if and only if there exists $X \in \mathbb{N}$ satisfying

$$X \leq H, \quad 2M \mid \rho(Z)^2 - X^2, \quad K \mid H^2 - X^2.$$

Every exponent used in a power of two or a prime power in the construction has a polynomial-time computable binary representation. The polynomial-space bit procedures for H and K are uniform in the clause oracle.

For

$$\alpha = (\alpha_0, \dots, \alpha_{\ell-1}) \in \{-1, +1\}^\ell,$$

let $\mathbf{u}(\alpha) \in \{0, 1\}^\ell$ be the assignment defined by

$$u_i(\alpha) = \frac{1 + \alpha_i}{2} \quad (0 \leq i < \ell).$$

Thus $\alpha_i = +1$ represents $u_i = 1$, and $\alpha_i = -1$ represents $u_i = 0$.

The proof is divided among Sections 5.1–5.4. In Section 5.1, we define a linear form $\mathcal{E}(\alpha)$, an integer τ_Z , and a power of two M , and prove that F_n has a satisfying assignment $\mathbf{u}$ with $\text{word}_L(\mathbf{u}) = Z$ if and only if

$$\mathcal{E}(\alpha) \equiv \tau_Z \pmod{M}$$

for some $\alpha \in \{-1, +1\}^\ell$. In Section 5.2, after writing $\mathcal{E}(\alpha) = \sum_{i=0}^{\ell-1} c_i \alpha_i$, we construct positive integers $\theta_0, \dots, \theta_{\ell-1}$, define $H = \sum_i \theta_i$, and prove that

$$|X| \leq H \quad \text{and} \quad K \mid H^2 - X^2$$

hold if and only if

$$X = \sum_{i=0}^{\ell-1} \alpha_i \theta_i$$

for some $\alpha \in \{-1, +1\}^\ell$; moreover, $\theta_i \equiv c_i \pmod{M}$. In Section 5.3, we replace the congruence modulo M by $2M \mid \rho(Z)^2 - X^2$. Finally, Section 5.4 proves that indexed bits of H and K are computable in polynomial space.

5.1 Combining All Exact-1-in-3 Clauses into One Congruence

For $0 \leq i < \ell$ and $0 \leq k < m$, let d_{ik} be the number of positive occurrences of u_i in clause k , minus the number of negative occurrences. Define

$$s_k(\alpha) = \sum_{i=0}^{\ell-1} d_{ik} \alpha_i.$$

Every true literal occurrence contributes $+1$ to this sum, and every false literal occurrence contributes -1 . If t_k is the number of true literal occurrences in clause k under $\mathbf{u}(\alpha)$, then

$$s_k(\alpha) = 2t_k - 3.$$

Consequently, clause k has exactly one true literal occurrence if and only if

$$s_k(\alpha) = -1. \tag{9}$$

Put

$$T = \sum_{k=0}^{m-1} 4^k = \frac{4^m - 1}{3}, \quad A = 4^m = 2^{2m}.$$

For $0 \leq Z < 2^L$, define

$$\mathcal{E}(\alpha) = \sum_{k=0}^{m-1} s_k(\alpha) 4^k + A \sum_{b=0}^{L-1} 2^b \alpha_b,$$

$$\tau_Z = -T + A(2Z - (2^L - 1)), \quad M = A2^{L+1}.$$

Define

$$\Delta(\alpha) = \sum_{k=0}^{m-1} (s_k(\alpha) + 1)4^k.$$

Since

$$\sum_{b=0}^{L-1} 2^b \alpha_b = 2 \text{word}_L(\mathbf{u}(\alpha)) - (2^L - 1),$$

the definitions give

$$\mathcal{E}(\alpha) - \tau_Z = \Delta(\alpha) + 2A(\text{word}_L(\mathbf{u}(\alpha)) - Z). \quad (10)$$

► **Lemma 29** (Congruence encoding of Exact-1-in-3 assignments). *For every integer Z with $0 \leq Z < 2^L$, the following are equivalent:*

- (i) F_n has a satisfying assignment $\mathbf{u}$ with $\text{word}_L(\mathbf{u}) = Z$;
- (ii) there exists $\alpha \in \{-1, +1\}^\ell$ such that

$$\mathcal{E}(\alpha) = \tau_Z;$$

- (iii) there exists $\alpha \in \{-1, +1\}^\ell$ such that

$$\mathcal{E}(\alpha) \equiv \tau_Z \pmod{M}.$$

Proof. For every k , the coefficient $(s_k(\alpha) + 1)/2$ belongs to $\{-1, 0, 1, 2\}$, a complete residue system modulo 4. The base-4 representation of $\Delta(\alpha)/2$ with digits in this set is unique. Consequently,

$$\Delta(\alpha) = 0 \iff s_k(\alpha) = -1 \quad (0 \leq k < m). \quad (11)$$

The same coefficient bound gives

$$|\Delta(\alpha)| \leq 4 \sum_{k=0}^{m-1} 4^k = \frac{4(A-1)}{3} < 2A. \quad (12)$$

By (10), equality $\mathcal{E}(\alpha) = \tau_Z$ is equivalent to

$$\Delta(\alpha) + 2A(\text{word}_L(\mathbf{u}(\alpha)) - Z) = 0.$$

The second term is a multiple of $2A$, while the first has absolute value less than $2A$. The equality therefore holds if and only if $\Delta(\alpha) = 0$ and $Z = \text{word}_L(\mathbf{u}(\alpha))$. By (11) and (9), this is precisely item (i). This proves the equivalence of (i) and (ii).

Finally,

$$|\mathcal{E}(\alpha) - \tau_Z| < 2A + 2A(2^L - 1) = A2^{L+1} = M.$$

An integer of absolute value less than M is divisible by M if and only if it is zero. This proves the equivalence of (ii) and (iii). ◀

For $0 \leq i < \ell$, let c_i be the coefficient of α_i in $\mathcal{E}$. Explicitly,

$$c_i = \sum_{k=0}^{m-1} d_{ik} 4^k + \begin{cases} A 2^i, & 0 \leq i < L, \\ 0, & L \leq i < \ell. \end{cases}$$

Thus

$$\mathcal{E}(\alpha) = \sum_{i=0}^{\ell-1} c_i \alpha_i.$$

By item (iii) of Lemma 29, F_n has a satisfying assignment $\mathbf{u}$ with $\text{word}_L(\mathbf{u}) = Z$ if and only if there exists $\alpha \in \{-1, +1\}^\ell$ such that

$$\sum_{i=0}^{\ell-1} c_i \alpha_i \equiv \tau_Z \pmod{M}. \quad (13)$$

5.2 Encoding the Assignment Vector by One Integer

Set

$$e = 2(m + \ell).$$

Let $p_0, \dots, p_{\ell-1}$ be the first ℓ odd primes, and define

$$K = \prod_{i=0}^{\ell-1} p_i^e, \quad P_i = K/p_i^e.$$

For each i , the integer P_i is odd and therefore invertible modulo the power of two M . Let $\lambda_i \in \{0, \dots, M-1\}$ be the unique integer satisfying

$$\lambda_i \equiv c_i P_i^{-1} \pmod{M}.$$

Define

$$t_i = \begin{cases} \lambda_i, & \lambda_i > 0, p_i \nmid \lambda_i, \\ \lambda_i + M, & \text{otherwise.} \end{cases}$$

Define

$$\theta_i = t_i P_i, \quad H = \sum_{i=0}^{\ell-1} \theta_i.$$

The definitions give

$$\begin{aligned} 0 < t_i < 2M, \quad \theta_i &\equiv c_i \pmod{M}, \\ p_j^e \mid \theta_i \quad (j \neq i), \quad p_i &\nmid \theta_i. \end{aligned} \quad (14)$$

Indeed, the congruence follows from $\theta_i = t_i P_i$, the divisibility for $j \neq i$ follows from $p_j^e \mid P_i$, and p_i divides neither t_i (by the case split, since $p_i \nmid M$) nor P_i .

We also need the following bound. Since $m, \ell \geq 1$, put $q = m + \ell \geq 2$. Using $L \leq \ell$, $\ell \leq 2^{\ell-1}$, and $M = 2^{2m+L+1}$, we obtain

$$4\ell M \leq 2^{2m+2\ell+2} = 4^{q+1} < 9^q = 3^e.$$

Since $p_i \geq 3$, (14) gives

$$\theta_i < 2MP_i \leq \frac{2MK}{3^e}.$$

Summing over $0 \leq i < \ell$ yields

$$2H < \frac{4\ell MK}{3^e} < K. \quad (15)$$

► **Lemma 30** (Recovering α from X). *For every integer X , the following are equivalent:*

- (i) $|X| \leq H$ and $K \mid H^2 - X^2$;
- (ii) there exists $\alpha \in \{-1, +1\}^\ell$ such that

$$X = \sum_{i=0}^{\ell-1} \alpha_i \theta_i.$$

Proof. Assume (ii). For each i ,

$$H - X = \sum_{j=0}^{\ell-1} (1 - \alpha_j) \theta_j, \quad H + X = \sum_{j=0}^{\ell-1} (1 + \alpha_j) \theta_j.$$

By (14),

$$\alpha_i = +1 \implies p_i^e \mid H - X, \quad \alpha_i = -1 \implies p_i^e \mid H + X.$$

The prime powers p_i^e are pairwise coprime, so $K \mid (H - X)(H + X) = H^2 - X^2$. The triangle inequality gives $|X| \leq H$. Thus (i) holds.

Conversely, assume (i). For every i ,

$$H \equiv \theta_i \not\equiv 0 \pmod{p_i},$$

because p_i divides θ_j for every $j \neq i$, but does not divide θ_i . Hence p_i cannot divide both $H - X$ and $H + X$: if it did, it would divide $2H$, and therefore H , since p_i is odd. Because $p_i^e \mid (H - X)(H + X)$, the whole power p_i^e divides exactly one of the two factors. Define

$$\alpha_i = \begin{cases} +1, & p_i^e \mid H - X, \\ -1, & p_i^e \mid H + X. \end{cases}$$

Let $X' = \sum_{i=0}^{\ell-1} \alpha_i \theta_i$. For every i , the definition of α_i and (14) give

$$X \equiv X' \pmod{p_i^e}.$$

Therefore $X \equiv X' \pmod{K}$. Both integers lie in $[-H, H]$, and $2H < K$ by (15); hence $X = X'$. Thus (ii) holds. ◀

5.3 Expressing the Assignment Congruence by Square Divisibility

For $\alpha \in \{-1, +1\}^\ell$, put

$$X = \sum_{i=0}^{\ell-1} \alpha_i \theta_i.$$

By (14),

$$X \equiv \sum_{i=0}^{\ell-1} \alpha_i c_i = \mathcal{E}(\alpha) \pmod{M}.$$

Thus (13) is equivalent to $X \equiv \tau_Z \pmod{M}$. The next lemma replaces this congruence by a condition involving X^2 . Its conclusion allows both residues τ_Z and $-\tau_Z$; in the proof of Proposition 32, the second residue is handled by replacing α with $-\alpha$.

► **Lemma 31** (Power-of-two square test). *Let $q \geq 1$ be an integer, put $Q = 2^q$, and let R be an odd integer. For every integer X ,*

$$2Q \mid R^2 - X^2 \iff X \equiv R \pmod{Q} \text{ or } X \equiv -R \pmod{Q}.$$

Proof. Suppose first that $X \equiv R \pmod{Q}$. Then $Q \mid R - X$. Since $q \geq 1$, the congruence implies that X is odd, and hence $R + X$ is even. Thus $2Q \mid (R - X)(R + X) = R^2 - X^2$. The case $X \equiv -R \pmod{Q}$ is identical.

Conversely, suppose $2^{q+1} \mid (R - X)(R + X)$. Since R is odd, X must be odd; otherwise both factors would be odd. The two factors are therefore even. Their difference is $2X$, which is not divisible by 4, so one of the two factors is divisible by 2 but not by 4. The other factor must then be divisible by $2^q = Q$. Hence $Q \mid R - X$ or $Q \mid R + X$. ◀

Put

$$\rho(Z) = M + \tau_Z.$$

Since $T < A$ and $0 \leq Z < 2^L$,

$$|\tau_Z| \leq T + A(2^L - 1) < A2^L = M/2.$$

Thus $\rho(Z) > 0$. The integer T is odd and the remaining term in τ_Z is even, so τ_Z , and therefore $\rho(Z)$, is odd. Also, $\rho(Z) \equiv \tau_Z \pmod{M}$.

► **Proposition 32** (Square-divisibility encoding with one integer). *For every integer Z with $0 \leq Z < 2^L$, the formula F_n has a satisfying assignment $\mathbf{u}$ with $\text{word}_L(\mathbf{u}) = Z$ if and only if there exists $X \in \mathbb{N}$ such that*

$$X \leq H, \quad 2M \mid \rho(Z)^2 - X^2, \quad K \mid H^2 - X^2.$$

Proof. Suppose that F_n has a satisfying assignment $\mathbf{u}$ with $\text{word}_L(\mathbf{u}) = Z$. By item (iii) of Lemma 29, choose $\alpha \in \{-1, +1\}^\ell$ such that

$$\sum_{i=0}^{\ell-1} c_i \alpha_i \equiv \tau_Z \pmod{M}.$$

Set

$$X_0 = \sum_{i=0}^{\ell-1} \alpha_i \theta_i.$$

By (14), $X_0 \equiv \tau_Z \equiv \rho(Z) \pmod{M}$. By Lemma 30, $|X_0| \leq H$ and $K \mid H^2 - X_0^2$. Since $\rho(Z)$ is odd, Lemma 31 gives $2M \mid \rho(Z)^2 - X_0^2$. Replacing X_0 with $X = |X_0|$ preserves both square conditions and gives a witness in $\mathbb{N}$.

Conversely, suppose $X \in \mathbb{N}$ satisfies the three displayed conditions. By Lemma 30, there exists $\alpha \in \{-1, +1\}^\ell$ such that

$$X = \sum_{i=0}^{\ell-1} \alpha_i \theta_i.$$

By Lemma 31, either $X \equiv \rho(Z) \pmod{M}$ or $X \equiv -\rho(Z) \pmod{M}$. Put $\beta = \alpha$ in the first case and $\beta = -\alpha$ in the second. Then

$$\sum_{i=0}^{\ell-1} \beta_i c_i \equiv \sum_{i=0}^{\ell-1} \beta_i \theta_i \equiv \rho(Z) \equiv \tau_Z \pmod{M}.$$

By item (iii) of Lemma 29, the assignment

$$u_i = \frac{1 + \beta_i}{2} \quad (0 \leq i < \ell)$$

satisfies F_n and has $\text{word}_L(\mathbf{u}) = Z$. ◀

5.4 Polynomial-Space Indexed-Bit Computation for H and K

The integers H and K may have exponentially many bits, so a polynomial-time reduction cannot include their binary expansions. By Lemma 8, it suffices to give polynomial-space procedures that, on input 1^n and a binary bit position, return the corresponding bit of H or K , together with an exponential bound on their bit lengths.

► **Proposition 33** (Polynomial-space indexed-bit computation of H and K). *For every uniformly succinct family $(F_n, L)_n$ in Definition 27, the integers $H(n)$ and $K(n)$ are **FPSPACE** functions of 1^n . Moreover, there is a fixed polynomial g such that, with $\mathcal{D}(n) = 2^{4g(n)+20}$,*

$$H, K, M, \rho(Z) < 2^{\mathcal{D}(n)}$$

for every Z with $0 \leq Z < 2^L$.

Proof. The proof is given in Appendix A. The appendix first proves that, given 1^n , an index i , and a bit position, the corresponding bit of $c_i \bmod M$ is computable in polynomial space from the clause oracle. It then gives polynomial-space indexed access to each p_i and computes indexed bits of K , P_i , θ_i , and H by evaluating uniform iterated-multiplication, division, modular-inverse, and addition circuits. The circuit-evaluation step is justified by Lemma 47, and the arithmetic circuits are those of Hesse, Allender, and Barrington [10, 11]. Axler's bound on the i th prime [1] yields the displayed size bound. The indexed-bit characterization in Lemma 8 then shows that $H(n)$ and $K(n)$ belong to **FPSPACE**. ◀

Proof of Theorem 28. The integers M, H, K and the affine function ρ were defined in the preceding subsections. The definitions give $M, H, K > 0$, and $|\tau_Z| < M/2$ gives $\rho(Z) = M + \tau_Z > 0$. Proposition 32 proves the stated equivalence for every $0 \leq Z < 2^L$, and Proposition 33 gives the polynomial-space procedures and size bounds for H and K . The powers of two used in the construction have exponents of the forms

$$L, \quad 2k, \quad 2m, \quad 2m + i, \quad 2m + L + 1,$$

where the relevant indices have polynomial-length binary representations. The exponent on each prime in K is $e = 2(m + \ell)$. All these exponents are computable in polynomial time from 1^n and the relevant index. This proves the remaining uniformity claim. ◀

The normal form in Theorem 28 contains the integers H and K , which are specified by polynomial-space indexed-bit procedures rather than by explicit binary expansions. It also contains the products X^2 , $\rho(Z)^2$, and H^2 . In the following corollary, Corollary 25 supplies polynomial-size EPAD definitions of H and K , and the quotient-sharing construction from Lemma 23 expresses the three products.

► **Corollary 34** (Succinct Exact-1-in-3 representation in EPAD). *From a uniformly succinct family $(F_n, L)_n$ as in Definition 27, one can construct in polynomial time an EPAD formula $\Xi_n(Z)$ such that, for every integer Z with $0 \leq Z < 2^L$,*

$$\Xi_n(Z) \iff F_n \text{ has a satisfying assignment } \mathbf{u} \text{ with } \text{word}_L(\mathbf{u}) = Z.$$

Proof. Defining the constants. By Proposition 33 and Corollary 25, apply the functional-EPAD construction to the zero-input functions with values $H(n)$ and $K(n)$. This gives polynomial-size EPAD formulas whose unique outputs are H and K . Using signed scaling from 1, introduce variables constrained by

$$A = 2^{2m}, \quad C = 2^{2m+L}, \quad M = 2^{2m+L+1}, \quad U_L = 2^L.$$

Introduce $T \geq 0$ with $3T + 1 = A$, and use signed scaling with exponent $2m$ to impose $Z_A = AZ$. Since $C - A = A(2^L - 1)$, impose

$$\rho = M - T - C + A + 2Z_A.$$

This equality gives $\rho = M + \tau_Z = \rho(Z)$. Finally, impose $0 \leq Z < U_L$.

Bounds used by the product construction. Put $\mathcal{D} = \mathcal{D}(n) = 2^{4g(n)+20}$, where g is the polynomial fixed in Proposition 33. The binary representation of $\mathcal{D}$ has polynomial length. For $0 \leq Z < 2^L$, $H, K, M, \rho(Z) < 2^{\mathcal{D}}$. Using signed scaling from 1, introduce variables constrained by

$$U_{\mathcal{D}} = 2^{\mathcal{D}}, \quad U_{2\mathcal{D}} = 2^{2\mathcal{D}}, \quad \Lambda = 2^{2\mathcal{D}+2}.$$

Expressing the squares and divisibilities. Impose $0 \leq \rho < U_{\mathcal{D}}$, quantify X with $0 \leq X \leq H$, and quantify $0 \leq Z_X, Z_\rho, Z_H < U_{2\mathcal{D}}$. Use three copies of the quotient-sharing product construction, with operand bound $2^{\mathcal{D}}$, product bound $2^{2\mathcal{D}}$, and scaling factor Λ , to impose

$$Z_X = X^2, \quad Z_\rho = \rho^2, \quad Z_H = H^2.$$

Add the divisibility atoms

$$2M \mid Z_\rho - Z_X, \quad K \mid Z_H - Z_X.$$

Let $\Xi_n(Z)$ be the existential closure of the conjunction of all constraints introduced in this proof, with Z as its only free variable. For each application of Lemma 23, the operand bound is $M_U = 2^{\mathcal{D}}$, the product bound is $M_W = 2^{2\mathcal{D}}$, and the scaling factor is $\Lambda = 2^{2\mathcal{D}+2}$. The inequality

$$2^{2\mathcal{D}+2} > 2^{\mathcal{D}} + 2^{2\mathcal{D}} + 1$$

therefore verifies the hypothesis $\Lambda > M_U + M_W + 1$ of that lemma. Every displayed multiplication by a power of two is represented by a scaling system, not by an explicit exponentially large coefficient. By Proposition 32, $\Xi_n(Z)$ holds exactly when F_n has a satisfying assignment $\mathbf{u}$ with $\text{word}_L(\mathbf{u}) = Z$. Its size is polynomial because all displayed exponents have polynomial-length binary representations and the formulas defining H and K have polynomial size. ◀

► **Lemma 35** (Local conversion from 3CNF to Exact-1-in-3). *Let G be a 3CNF formula with m clauses and ℓ variables. One can construct in polynomial time an Exact-1-in-3 formula F containing every variable of G such that every assignment to the variables of G extends to a satisfying assignment of F if and only if it satisfies G . The formula F has $3m$ clauses and $\ell + 4m$ variables. For a family of 3CNFs given by a polynomial-time clause oracle, the resulting Exact-1-in-3 family also has a polynomial-time clause oracle, and the variables inherited from the 3CNF retain their original indices.*

Proof. Write $R(x, y, z)$ for the requirement that exactly one of the three displayed literal occurrences is true. For each clause $x \vee y \vee z$ of G , introduce four fresh variables a, b, c, d and the three clauses

$$R(\neg x, a, b), \quad R(y, b, c), \quad R(\neg z, c, d).$$

This is the classical local Exact-1-in-3 gadget of Schaefer [21]. Fix truth values $x, y, z \in \{0, 1\}$ for the three literals. The clauses are equivalent to

$$a + b = x, \quad b + c = 1 - y, \quad c + d = z.$$

If $y = 1$, take $b = c = 0$, $a = x$, and $d = z$. If $y = 0$ and $x = 1$, take $b = 1$, $a = c = 0$, and $d = z$. If $x = y = 0$, the first two equations force $b = 0$ and $c = 1$, so the third equation is solvable exactly when $z = 1$. Thus the gadget has a satisfying extension exactly when $x \vee y \vee z$ is true.

For the succinct statement, an output clause index k determines the original clause $\lfloor k/3 \rfloor$ and one of its three gadget clauses. The oracle recovers that clause and computes the four fresh-variable indices by binary arithmetic. These operations preserve the indices of the original variables and take polynomial time in the index length. ◀

6 NEXP-Completeness of EPAD Satisfiability

Proof of Theorem 1. Membership follows from the exponential small-solution theorem of Lechner, Ouaknine, and Worrell [14]. For hardness, let $\mathcal{L} \in \mathbf{NEXP}$. The standard succinct Cook–Levin construction gives a family of 3CNFs $(G_n)_n$ whose first n variables are the input variables and such that, for every $\mathbf{a} \in \{0, 1\}^n$,

$$\mathbf{a} \in \mathcal{L} \iff G_n \text{ has a satisfying assignment extending } \mathbf{a}.$$

The remaining variables describe an accepting exponential-time tableau, and a polynomial-time oracle recovers each local consistency clause from its polynomial-length index; see Papadimitriou and Yannakakis [20] or Papadimitriou [19, Section 20.1].

Apply Lemma 35 to obtain a uniformly succinct Exact-1-in-3 family $(F_n)_n$. The variables inherited from G_n , including the first n input variables, retain their indices. Thus

$$\mathbf{a} \in \mathcal{L} \iff F_n \text{ has a satisfying assignment } \mathbf{u} \text{ with } u_i = a_i \text{ for } 0 \leq i < n.$$

Apply Corollary 34 with $L = n$. For the concrete input $\mathbf{a} = (a_0, \dots, a_{n-1})$, substitute

$$Z_{\mathbf{a}} = \sum_{i=0}^{n-1} a_i 2^i$$

for the free variable Z of the EPAD formula. The resulting polynomial-size EPAD sentence is satisfiable exactly when $\mathbf{a} \in \mathcal{L}$. The Exact-1-in-3 clause oracle and the EPAD formula are constructed in polynomial time, so this is a polynomial-time many-one reduction. Hence EPAD satisfiability is **NEXP**-hard. ◀

7 NEXP-Complete Finite-Quotient Normalization and Exponential Coefficient Growth

Finite-quotient normalization replaces a divisibility atom by equations for its possible integer quotients. The definitions below specify the replacement step and a class of systems for which repeated replacements remove every divisibility atom. A polynomial-size family shows that a specified replacement sequence may produce coefficients with exponentially many bits.

► **Definition 36** (Finite-quotient replacement). *For an atom $L \mid M$ on a branch, a finite-quotient replacement with quotient set $\mathcal{Q} \subseteq \mathbb{Z}$ is valid if the current branch constraints, including the remaining divisibility atoms, together with $L \mid M$ force either $L = M = 0$, or $L \neq 0$ and $M/L \in \mathcal{Q}$. It creates one child for each equation $M = cL$, with $c \in \mathcal{Q}$.*

The case $L = M = 0$ satisfies every child equation. We make a child equation primitive by dividing the coefficients of $M - cL = 0$ by their greatest common divisor. A branch closes when the accumulated constraints are inconsistent. A system reduces to divisibility-free branches if a sequence of such replacements leaves every branch either closed or without divisibility atoms.

The completeness constructions below require an explicit bound on the size of satisfying assignments; the following theorem supplies it.

► **Theorem 37** (Effective small-solution bound; Theorem 14 of [14]). *There is a computable polynomial p such that every satisfiable conjunction of affine equalities, strict affine inequalities, and affine divisibility atoms of size n has an integer solution in which every variable has bit length at most $2^{p(n)}$.*

7.1 A Merge-Absorptive Fragment with Complete Finite-Quotient Elimination

Défossez, Haase, Mansutti, and Pérez study divisibility systems with the following property: the variables admit an order such that, among all nontrivial divisibilities entailed by the system, the greatest variable with respect to that order occurs only on the right-hand side [5]. For the systems considered there, feasibility under this restriction is in **NP**. The condition introduced here proceeds instead by contraction. An eligible atom has only finitely many possible quotients and joins two previously separate variable components. We call a system for which such atoms can successively connect all components *merge-absorptive*, as formalized below.

Recall that a positive homogeneous divisibility system consists of divisibility atoms between homogeneous integer linear forms, together with homogeneous linear side conditions and positivity constraints. The support of a linear form is the set of variables with nonzero coefficient. A valuation of a component C is *admissible* if it is positive and satisfies all side conditions supported on C . A form supported on C is *nonvanishing* on C if it is nonzero on every admissible valuation; positivity and negativity are defined analogously.

A homogeneous equality $F = 0$ is identified with the divisibility atom $0 \mid F$. Thus equalities may serve as linear merge atoms.

For disjoint variable sets A, B , a form R_B supported on B is *side-condition bounded by A* if every variable in its support is bounded above on the real polyhedron given by the side conditions on $A \cup B$, positivity, and the normalization $\sum_{x \in A} x = 1$.

Fix a partition Π of the variables, and let A, B be distinct components. Decompose every form supported on $A \cup B$ as $F = F_A + F_B$. An atom $L \mid M$ is a *merge atom* for A, B if it mentions both components and, after possibly exchanging them, satisfies one of the following conditions.

1. *Linear merge*: $L = 0$ and $M = F_A + F_B$, where F_A and F_B are nonvanishing on their components.
2. *Bounded-divisor merge*: $L = P_A + R_B$ and $M = Q_A$, where P_A, R_B have the same strict sign on their components and Q_A is nonvanishing on A .
3. *Bounded-dividend merge*: $L = P_A$ and $M = T_A + R_B$, where P_A is nonvanishing on A , R_B is nonvanishing on B , and R_B is side-condition bounded by A .

► **Definition 38** (Merge-absorptive systems). *A positive homogeneous divisibility system is merge-absorptive if, starting from the singleton partition, some sequence of merge atoms brings all variables into one component.*

► **Remark 39** (Comparison with ordered divisibility systems). The ordered restriction described above and merge-absorptivity are incomparable. The system

$$x \mid y, \quad x > 0, \quad y > 0,$$

satisfies the ordered restriction for $x \prec y$, but is not merge-absorptive because the quotient y/x is unbounded. Conversely, Δ_1 is merge-absorptive by Lemma 40, but does not satisfy the ordered restriction. In its atom

$$y_1 - 2y_0 \mid y_1,$$

the order $y_0 \prec y_1$ places the greatest variable on both sides, whereas the order $y_1 \prec y_0$ places it only on the left. Thus one restriction organizes divisibility dependencies in a forward direction, while the other contracts variable components through bounded quotient relations.

Examples 49 and 50 in Appendix B illustrate a finite-quotient replacement and a two-step sequence of merge atoms.

► **Lemma 40** (Scaling systems are merge-absorptive). *The systems in Lemmas 19 and 21 are merge-absorptive.*

Proof. For $t = 0$, the equality $r = x$, identified with $0 \mid r - x$, is a linear merge. For $t \geq 1$, process the atoms (3) for $i = t, t - 1, \dots, 1$. Once y_t, y_i are in one component, the atom

$$y_i + y_{i-1} \mid y_t + 3y_i$$

is a bounded-divisor merge with the singleton $\{y_{i-1}\}$. A positive-scaling chain consists of these systems joined by homogeneous equalities, each identified with a linear merge atom and merging the next component. ◀

► **Proposition 41** (Recognition and normalization). *For positive homogeneous divisibility systems, merge-absorptive recognition is decidable in polynomial time, and every merge-absorptive system reduces to divisibility-free branches.*

Proof sketch. After normalizing each component by setting the sum of its variables to one, linear programming decides positivity, nonvanishing, and side-condition boundedness. A union-find algorithm applies eligible atoms; eligibility is monotone under component coarsening. On each branch, all variables in a component are rational multiples of one positive parameter. A merge atom has finitely many possible integer quotients and either closes the branch or relates the parameters of two components. Once one component remains, each residual atom reduces to an integer divisibility test. The proof is given in Appendix B. ◀

An *affine divisibility system* is a conjunction of affine-linear equalities, strict affine-linear inequalities, and divisibility atoms between affine-linear terms over integer variables.

► **Proposition 42** (EPAD to affine divisibility). *Every EPAD formula has a polynomial-time equisatisfiable translation to an affine divisibility system over integer variables.*

Proof sketch. Rewrite the matrix as a positive Boolean combination and use Theorem 37 to bound all arithmetic values. Boolean indicator variables and bounded products from Lemma 23 remove the Boolean connectives. Explicit operand and product bounds determine a power of two $\Lambda > M_U + M_W + 1$ for every product. The detailed construction is in Appendix B. ◀

► **Proposition 43** (EPAD to merge-absorptive systems). *Every EPAD formula has a polynomial-time equisatisfiable translation to a merge-absorptive positive homogeneous divisibility system.*

Proof sketch. First apply Proposition 42. Replace signed variables by differences of positive variables and homogenize constants with a fresh positive scale u . By Theorem 37, the required bound has a polynomial-length exponent E , and Lemma 21 imposes $s_0 = 2^E u$. Represent the original variables by successive quotients $q_j = (s_{j-1} - s_j)/u$. The scaling-system atoms first merge their auxiliary variables. The quotient atoms then merge $s_1, s_2, \dots$ successively into the same component. The equisatisfiability proof is given in Appendix B. ◀

► **Corollary 44** (NEXP-completeness). *Satisfiability for merge-absorptive positive homogeneous divisibility systems is NEXP-complete under polynomial-time many-one reductions.*

Proof. Hardness follows from Theorem 1 and Proposition 43. Membership holds because the fragment is a syntactic fragment of EPAD. ◀

Finite-quotient replacement eliminates all divisibility atoms in this fragment, but the equations produced along a replacement sequence need not have polynomial-size coefficients. The multiplier systems give an exponential example.

7.2 A Family Forcing Exponentially Large Primitive Coefficients

Recall that $\Delta_j(x, r)$ is polynomial-size and merge-absorptive, and forces $r = (2^{2^j} - 1)x$.

► **Lemma 45** (Quotient at each stage). *For $j \geq 1$, consider the finite-quotient replacement sequence for Δ_j that processes the merge atoms in the order from Lemma 40. At each stage $i = j, j-1, \dots, 1$, the only satisfiable child adds the primitive equation*

$$y_j + 3y_i = (2^{2^{j-i}} + 1)(y_i + y_{i-1}).$$

Proof. Put $h = j - i$ and $B = 2^{2^h}$. The stage invariant and forced ratio are

$$y_j = (B - 1)y_i, \quad y_i = (B + 1)y_{i-1}.$$

Consequently,

$$\frac{y_j + 3y_i}{y_i + y_{i-1}} = \frac{(B + 2)(B + 1)y_{i-1}}{(B + 2)y_{i-1}} = B + 1.$$

Every child with another quotient is inconsistent. The surviving equation is

$$y_j + (2 - 2^{2^h})y_i - (2^{2^h} + 1)y_{i-1} = 0.$$

It is primitive because the coefficient of y_j is 1. At $i = 1$, the retained coefficient has bit length $\Theta(2^j)$. ◀

► **Theorem 46** (Large coefficients in finite-quotient replacements). *The family (Δ_j) is polynomial-size and is merge-absorptive. Processing its merge atoms in the order from Lemma 40 and applying finite-quotient replacements produces primitive equations with coefficient bit length $\Theta(2^j)$. The same bit length is necessary for any primitive equation over x, r defining the (x, r) -projection.*

Proof. Polynomial size follows from Lemma 19, and membership in the merge-absorptive fragment follows from Lemma 40. By Lemma 45, the specified replacement sequence produces $2^{2^j-1} + 1$, whose bit length is $\Theta(2^j)$.

Put $M_j = 2^{2^j} - 1$. The projected solution set is exactly

$$\{(x, M_j x) : x \in \mathbb{N}_{>0}\}.$$

Any nonzero affine equation valid on this ray and involving only x, r has zero constant term and is an integer multiple of $r - M_j x = 0$. Its primitive representatives are $\pm(r - M_j x)$, and therefore contain a coefficient of absolute value M_j , with bit length $\Theta(2^j)$. ◀

8 Summary of Results

We proved that EPAD satisfiability is **NEXP**-hard; combined with the known **NEXP** upper bound, this gives **NEXP**-completeness. We also proved **ALC** = **FPSPACE** and **succALC** = **FEXP**, with both classes contained in **FEPAD**. The lower bound uses fixed-width arithmetic logic circuits, functional EPAD definitions of their gates, and a square-divisibility encoding of succinct Exact-1-in-3 formulas that represents the values of a specified initial sequence of variables by an integer parameter. We also defined the merge-absorptive fragment and proved **NEXP**-hardness for its satisfiability problem; the known upper bound again gives **NEXP**-completeness, although finite-quotient replacement eliminates all divisibility atoms. For a polynomial-size multiplier family, a specified replacement sequence produces primitive equations with coefficients of exponential bit length. Every primitive affine equation over the distinguished variables that defines the same projected relation also has a coefficient of exponential bit length.

References

- 1 Christian Axler. New estimates for the n th prime number. *Journal of Integer Sequences*, 22:Article 19.4.2, 2019.
- 2 A. P. Belyukov. Decidability of the universal theory of natural numbers with addition and divisibility. *Journal of Soviet Mathematics*, 14(5):1436–1444, 1980. doi:10.1007/BF01693974.
- 3 Daniel Bundala and Joël Ouaknine. Advances in parametric real-time reasoning. In *Mathematical Foundations of Computer Science 2014*, volume 8634 of *Lecture Notes in Computer Science*, pages 123–134. Springer, 2014. doi:10.1007/978-3-662-44522-8_11.
- 4 Joel D. Day and Matthew Konefal. Word equations with length constraints via weak arithmetics and matrix reachability problems. In *Reachability Problems*, volume 16230 of *Lecture Notes in Computer Science*, pages 51–67. Springer, 2025. doi:10.1007/978-3-032-09524-4_4.
- 5 Rémy Défossez, Christoph Haase, Alessio Mansutti, and Guillermo A. Pérez. Integer programming with GCD constraints. In *Proceedings of the 2024 Annual ACM-SIAM Symposium on Discrete Algorithms*, SODA 2024, pages 3605–3658, 2024. doi:10.1137/1.9781611977912.128.
- 6 Matthias Galota and Heribert Vollmer. Functions computable in polynomial space. *Information and Computation*, 198(1):56–70, 2005. doi:10.1016/j.ic.2004.10.004.
- 7 Hana Galperin and Avi Wigderson. Succinct representations of graphs. *Information and Control*, 56(3):183–198, 1983.

- 8 Christoph Haase. *On the Complexity of Model Checking Counter Automata*. PhD thesis, University of Oxford, 2012. URL: <https://www.cs.ox.ac.uk/files/4759/thesis.pdf>.
- 9 Christoph Haase. A survival guide to Presburger arithmetic. *ACM SIGLOG News*, 5(3):67–82, 2018.
- 10 William Hesse, Eric Allender, and David A. Mix Barrington. Uniform constant-depth threshold circuits for division and iterated multiplication. *Journal of Computer and System Sciences*, 65(4):695–716, 2002. doi:10.1016/S0022-0000(02)00025-9.
- 11 William Hesse, Eric Allender, and David A. Mix Barrington. Corrigendum to “uniform constant-depth threshold circuits for division and iterated multiplication”. *Journal of Computer and System Sciences*, 80(2):496–497, 2014.
- 12 Richard E. Ladner. Polynomial space counting problems. *SIAM Journal on Computing*, 18(6):1087–1097, 1989. doi:10.1137/0218073.
- 13 Antonia Lechner. *Extensions of Presburger Arithmetic and Model Checking One-Counter Automata*. PhD thesis, University of Oxford, 2016. doi:10.5287/ora-nokxwrdnn.
- 14 Antonia Lechner, Joël Ouaknine, and James Worrell. On the complexity of linear arithmetic with divisibility. In *Proceedings of the 30th Annual ACM/IEEE Symposium on Logic in Computer Science*, LICS 2015, pages 667–676, 2015. doi:10.1109/LICS.2015.67.
- 15 Anthony W. Lin and Rupak Majumdar. Quadratic word equations with length constraints, counter systems, and Presburger arithmetic with divisibility. *Logical Methods in Computer Science*, 17(4):4:1–4:20, 2021. doi:10.46298/LMCS-17(4:4)2021.
- 16 Leonard Lipshitz. The Diophantine problem for addition and divisibility. *Transactions of the American Mathematical Society*, 235:271–283, 1978. doi:10.1090/S0002-9947-1978-0469886-1.
- 17 M. Lothaire. *Combinatorics on Words*. Cambridge Mathematical Library. Cambridge University Press, 1997.
- 18 Kenneth L. Manders and Leonard Adleman. NP-complete decision problems for binary quadratics. *Journal of Computer and System Sciences*, 16(2):168–184, 1978.
- 19 Christos H. Papadimitriou. *Computational Complexity*. Addison-Wesley, 1994.
- 20 Christos H. Papadimitriou and Mihalis Yannakakis. A note on succinct representations of graphs. *Information and Control*, 71(3):181–185, 1986.
- 21 Thomas J. Schaefer. The complexity of satisfiability problems. In *Proceedings of the Tenth Annual ACM Symposium on Theory of Computing*, pages 216–226, 1978. doi:10.1145/800133.804350.

A

 Polynomial-Space Indexed-Bit Algorithms for H and K

This appendix proves the indexed-bit claims in Proposition 33.

► **Lemma 47** (Evaluation of uniform circuits on indexed inputs). *Let z_n be a binary string of length $L(n) \leq 2^{\text{poly}(n)}$ presented by a **PSPACE** predicate that, on $(1^n, i)$, returns its i th bit. Let $(D_L)_L$ be a logspace-uniform circuit family of size $L^{O(1)}$ and depth $(\log L)^{O(1)}$. Gates are Boolean gates or unweighted threshold gates, where a threshold gate outputs one if and only if at least its specified number of inputs are one. Then every indexed output bit of $D_{L(n)}(z_n)$ is computable in polynomial space.*

Proof. Use recursive recomputation as in Proposition 9. Recursively evaluate the requested output gate. Gate indices, input indices, child-index counters, and threshold counters use $O(\log L) = \text{poly}(n)$ bits. For an unbounded-fan-in gate, enumerate all possible child indices. The logspace-uniformity algorithm decides whether a proposed index is a child of the current gate; recursively evaluate the actual children and count the true inputs. The recursion depth is $(\log L)^{O(1)} = \text{poly}(n)$. Recomputing a subgate affects time but not space. ◀

► **Lemma 48** (Inverse modulo a power of two). *Given an odd q -bit integer a , its inverse modulo 2^q is computed by a logspace-uniform polynomial-size threshold circuit of depth $O(\log q)$.*

Proof. Start with $u = 1 \pmod{2}$. If $au \equiv 1 \pmod{2^b}$, put $b' = \min(2b, q)$ and

$$u' = u(2 - au) \pmod{2^{b'}}.$$

Then $au' = 1 - (au - 1)^2 \equiv 1 \pmod{2^{b'}}$. There are $O(\log q)$ precision-doubling stages. By the uniform division circuits of Hesse, Allender, and Barrington [10, 11], uniform constant-depth threshold circuits perform multiplication and the underlying binary addition. Subtraction is implemented by two's-complement addition, while retaining the least significant b' bits is a wiring operation. Composing the stages gives polynomial size, depth $O(\log q)$, and logspace uniformity. ◀

Detailed proof of Proposition 33. Clause indices, variable indices, prime indices, and bit positions all have polynomial length in n .

Coefficient residues. Given a variable index i and a clause index k , evaluate the clause oracle and compute

$$d_{ik} = \#\{\text{positive occurrences of } u_i \text{ in clause } k\} - \#\{\text{negative occurrences of } u_i \text{ in clause } k\}.$$

Thus $d_{ik} \in \{-3, -2, -1, 0, 1, 2, 3\}$. The indexed bits of $d_{ik}4^k \pmod{M}$ are computable in polynomial space, since $4^k = 2^{2k}$ and d_{ik} has constant binary length.

For fixed i , give a uniform iterated-addition circuit indexed access to the m residues $d_{ik}4^k \pmod{M}$. If $i < L$, include the additional term $A2^i = 2^{2m+i}$. Retaining the least significant $\log_2 M$ bits of the sum computes

$$c_i \pmod{M}.$$

By Lemma 47, every indexed bit of this residue is computable in polynomial space.

Primes and K . Given i , enumerate odd candidates beginning with 3, test each candidate for primality by trial division, and count primes until reaching p_i . This may take exponential time, but the candidate, divisor, and counter have polynomially many bits, so the procedure uses polynomial space. Give the uniform iterated-multiplication circuit of Hesse, Allender,

and Barrington [10, 11] indexed access to a list containing e copies of each p_i . By Lemma 47, this gives polynomial-space indexed-bit access to

$$K = \prod_{i=0}^{\ell-1} p_i^e.$$

The integers θ_i and H . For fixed i , iterated multiplication gives p_i^e , and uniform division gives $P_i = K/p_i^e$. Take the least significant $\log_2 M$ bits of P_i , apply Lemma 48, multiply by $c_i \bmod M$, and retain the least significant $\log_2 M$ bits. This gives the integer $\lambda_i \in \{0, \dots, M-1\}$ satisfying

$$\lambda_i \equiv c_i P_i^{-1} \pmod{M}.$$

To decide whether $p_i \mid \lambda_i$, scan the bits of λ_i from most significant to least significant while maintaining the current residue modulo p_i . The definition in the main proof then gives

$$t_i = \begin{cases} \lambda_i, & \lambda_i > 0, p_i \nmid \lambda_i, \\ \lambda_i + M, & \text{otherwise.} \end{cases}$$

Uniform multiplication gives $\theta_i = t_i P_i$, and a uniform balanced tree of additions gives $H = \sum_i \theta_i$. Applying Lemma 47 to these circuits gives polynomial-space procedures for the individual bits of H and K .

For the size bound, fix a polynomial $g \geq 1$ dominating the binary lengths of m , ℓ , and L , and the running time of the clause oracle. Then

$$m, \ell \leq 2^{g(n)}, \quad e = 2(m + \ell) \leq 2^{g(n)+2}.$$

By enlarging g by a fixed additive constant, Axler's bound on the i th prime [1] gives $p_i < 2^{3g(n)+10}$ for every $i < \ell$. Set $\mathcal{D}(n) = 2^{4g(n)+20}$. Then

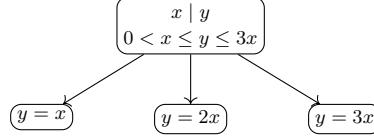
$$\log_2 K < (2^{g(n)})(2^{g(n)+2})(3g(n) + 10) < \mathcal{D}(n).$$

Moreover, the exponent defining M is at most $2 \cdot 2^{g(n)} + 2^{g(n)} + 1 < 2^{g(n)+2}$, so $M < 2^{\mathcal{D}(n)}$. By (15), $H < K < 2^{\mathcal{D}(n)}$. For $0 \leq Z < 2^L$, one has $0 < \rho(Z) < 2M < 2^{\mathcal{D}(n)}$. The binary representations of $\mathcal{D}(n)$, $2\mathcal{D}(n)$, and $2\mathcal{D}(n) + 2$ have polynomial length. Thus the indexed-bit characterization in Lemma 8 applies. $\blacktriangleleft$

B Proof of the Merge-Absorptive Reduction

The following examples illustrate the finite-quotient and merge steps used in the main argument.

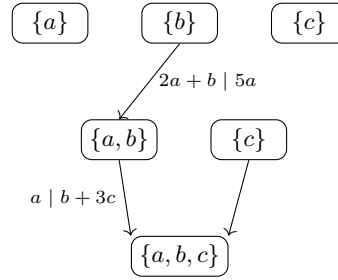
► **Example 49** (A finite-quotient replacement). Under the side conditions $0 < x$ and $x \leq y \leq 3x$, the quotient y/x belongs to $\{1, 2, 3\}$. The corresponding finite-quotient replacement has the three children shown below.



The three child branches add, respectively, $x - y = 0$, $2x - y = 0$, and $3x - y = 0$. Equivalently, they impose $y = x$, $y = 2x$, and $y = 3x$. Each child replaces the divisibility atom by one possible quotient equation. ◁

► **Example 50** (A two-step sequence of merge atoms). Take variables a, b, c over $\mathbb{N}_{>0}$, side conditions $a > b > c$, and the two divisibility atoms $2a + b \mid 5a$ and $a \mid b + 3c$.

The first atom merges $\{a\}$ and $\{b\}$: since $a > b$, the quotient $5a/(2a + b)$ lies strictly between $5/3$ and $5/2$, so it equals 2 and forces $a = 2b$. The second atom then merges $\{c\}$ into $\{a, b\}$: using $a = 2b$ and $b > c$, the quotient $(b + 3c)/a$ lies strictly between $1/2$ and 2, so it equals 1 and forces $b = 3c$. Thus all variables lie in one component, and the solutions that remain form the one-parameter set $(a, b, c) = (6s, 3s, s)$, $s \in \mathbb{N}_{>0}$.



The diagram indicates these two merge steps. ◁

Detailed proof of Proposition 41. For a component $C = \{z_1, \dots, z_r\}$, positivity and nonvanishing of a form F are linear-programming tests. Fix the scale by adding $z_1 + \dots + z_r = 1$, which selects one representative from each positive scaling class. Test strict feasibility by adding a common slack $\sigma > 0$ and requiring every strict side condition supported on C , as well as every constraint $z_i > 0$, to hold with margin σ . Homogeneous equalities are kept as equalities. Then F is nonvanishing if and only if $F = 0$ is infeasible, and positive if and only if $F \leq 0$ is infeasible. Homogeneity lets rational feasible points be scaled to positive integer valuations, so the rational and integer tests agree.

The side-condition boundedness test in the third merge condition is also a linear-programming test: for each variable y in the support of R_B , add $\sum_{x \in A} x = 1$ and the side conditions supported on $A \cup B$, then check whether y has finite supremum over the resulting polyhedron.

The recognizer uses union-find for the current partition. It repeatedly scans all atoms; when an atom mentions exactly two current components, it tests the three merge conditions in both orientations and unions the components after a successful test. If a scan performs no union, the fixed point has been reached: accept if one component remains, and reject otherwise. There are at most linearly many successful unions, and each scan uses polynomially many linear-programming tests.

The greedy merge procedure is complete because merge eligibility is monotone under coarsening. Positivity and nonvanishing persist when components are merged: admissible valuations of the larger component project to a subset of the old admissible valuations. Side-condition boundedness is monotone for the same reason. If $y \leq M \sum_{x \in A} x$, then after enlarging A the same valuation satisfies $y \leq M \sum_{x \in A'} x$, and added side conditions only restrict the feasible set. Thus performing one available merge never destroys another available merge.

To reach divisibility-free branches, maintain the invariant that, on every branch not closed by inconsistency, all variables in each current component are rational multiples of one positive parameter for that component. For a singleton component, its variable is such a parameter. A merge atom of the first kind gives a nontrivial linear equation between the two parameters and either closes the branch or expresses one parameter as a rational multiple of the other.

For the second kind, write on the current branch $P_A = pt_A$, $R_B = rt_B$, and $Q_A = qt_A$. Positivity of $P_A + R_B$ gives $p, r \neq 0$ with the same sign, and nonvanishing of Q_A gives $q \neq 0$. Then the quotient $Q_A/(P_A + R_B)$ has absolute value at most $|q/p|$. Hence there are only finitely many possible integer quotients c , and this finite set is obtained by listing the integers in that interval. Branching over them and adding $c(P_A + R_B) = Q_A$ either closes the branch or gives a nontrivial linear equation between t_A and t_B , thereby preserving the rational-multiple invariant after the two components are merged. This is the first merge of Example 50, where $c = 2$ and the added equality is $a = 2b$.

For the third kind, write $P_A = pt_A$, $T_A = st_A$, and $R_B = rt_B$, with $p, r \neq 0$. Side-condition boundedness gives a constant M such that R_B is bounded in absolute value by a fixed rational multiple of Mt_A on the current branch. Hence $(T_A + R_B)/P_A$ has only finitely many possible integer values; the relevant finite set is computed from this rational bound. Branching over these values again either closes the branch or adds a nontrivial linear equation between t_A and t_B , so the merged component again has one positive parameter. This is the second merge of Example 50, where the bound gives $c = 1$ and the added equality is $b = 3c$.

After all successful merges, every branch not closed by inconsistency has one component. Every remaining atom is then of the form $\lambda t \mid \mu t$ for rational constants λ, μ , with $t > 0$ on the branch. Cancellation by t is legitimate. Although λ and μ may be rational after solving branch equations, the original forms are integer-valued on integer solutions, so the atom is equivalent on the branch to a tautology, to an inconsistent $0 \mid \mu t$ with $\mu \neq 0$, or to the constant divisibility test $\mu/\lambda \in \mathbb{Z}$. Thus the branch reaches either inconsistency or an empty divisibility system. ◀

Detailed proof of Proposition 42. Put the quantifier-free matrix in negation normal form. Negated affine atoms become positive Boolean combinations of affine equalities and strict affine inequalities. Negated divisibility atoms can also be rewritten locally as in [14, Sec. IV.A]. Thus, after adding only polynomially many arithmetic variables, the formula is a positive Boolean combination of affine equalities, strict affine inequalities, and unnegated affine divisibility atoms.

We first bound all arithmetic variables. If the positive Boolean matrix is satisfiable, choose a conjunction of atoms along a satisfied branch. By Theorem 37, that conjunction has a bounded solution; because the matrix is positive, the same solution satisfies the whole matrix. Use the instance size for n in the bound. Introduce a variable B and force $B = 2^{2^{p(n)}+1}$ by $\text{Scale}_{2^{p(n)}+1}(1, B)$. Add $-B < x_i < B$ for every original and auxiliary arithmetic variable. For every affine term and product variable introduced in the construction, add a variable constrained to equal a power of two that bounds its absolute value. The exponents have

polynomial bit length, so all scaling systems are polynomial-size conjunctions of affine divisibility constraints and strict affine inequalities.

It remains to remove the Boolean structure. Give each subformula φ a Boolean variable b_φ , constrained by the strict inequalities $b_\varphi + 1 > 0$ and $2 - b_\varphi > 0$. For conjunctions and disjunctions, introduce a product variable p , add $-2 < p < 2$, and apply Lemma 23 with $U = b_\psi$, $V = b_\theta$, $W = p$, $M_U = M_W = 2$, and the constant $\Lambda = 8 > 2 + 2 + 1$. This enforces $p = b_\psi b_\theta$. Add $b_{\psi \wedge \theta} = p$ or $b_{\psi \vee \theta} = b_\psi + b_\theta - p$. For an atomic formula α , it is enough to enforce $b_\alpha = 1 \Rightarrow \alpha$. Because the syntax tree is positive and the variable of the whole matrix is forced to be 1, these one-way implications force a satisfying assignment for the original formula; reverse implications are unnecessary for equisatisfiability. If α is $h = 0$, introduce $T_\alpha = b_\alpha h$ and require $T_\alpha = 0$. If α is $h > 0$, introduce $T_\alpha = b_\alpha h$ and require $T_\alpha - b_\alpha + 1 > 0$. If α is $L \mid M$, introduce $L_\alpha = b_\alpha L$ and $M_\alpha = b_\alpha M$, and require $L_\alpha \mid M_\alpha$.

For each product $W = b_\alpha V$ introduced to enforce an implication, the bounds fixed above provide an explicit power-of-two bound M_W for W ; we write the side conditions $-M_W < W < M_W$. Apply Lemma 23 with $U = b_\alpha$, $M_U = 2$, and a power of two $\Lambda > M_U + M_W + 1$. The exponent of Λ has polynomial bit length, and every multiple by Λ is represented by fresh variables constrained through Lemma 22. This gives the explicit operand and product bounds required by the gadget for T_α , L_α , and M_α . Require the variable of the whole matrix to be 1.

If the original EPAD formula is satisfiable, choose a bounded satisfying assignment, set each Boolean variable to the truth value of its subformula, and interpret the product variables as the corresponding products. All constraints above are then satisfied. Conversely, a satisfying assignment of the affine-divisibility conjunction makes the Boolean output 1. Since every atom marked true is actually true, the original positive Boolean matrix is true under the arithmetic assignment. Hence the construction is equisatisfiable. $\blacktriangleleft$

Detailed proof of Proposition 43. By Proposition 42, reduce an affine divisibility system over integer variables. Replace each x by $x^+ - x^-$, with $x^+, x^- \in \mathbb{N}_{>0}$. Write $h = 0$ as $0 \mid h$, and $h > 0$ as $0 \mid h - s$ with fresh $s \in \mathbb{N}_{>0}$. This gives an equisatisfiable affine divisibility system $\mathcal{I}$ over positive integer variables.

Write the atoms of $\mathcal{I}$ as $L(\mathbf{x}) \mid M(\mathbf{x})$, where L, M are affine-linear forms with integer coefficients. If rational affine forms arise, multiplying both terms of an atom by a common positive denominator gives an equivalent atom with integer coefficients.

Apply Theorem 37. Let $x_1, \dots, x_k$ be the variables of $\mathcal{I}$, and let $N = 2^{p(|\mathcal{I}|)}$ be the resulting bit bound. If $\mathcal{I}$ is satisfiable, then it has a solution with $x_j < 2^N$ for every j . Put

$$E = N + \lceil \log_2(k+1) \rceil + 1.$$

Then $k2^N < 2^E$, and the binary length of E is polynomial in $|\mathcal{I}|$.

We construct a positive homogeneous system. Introduce u and variables s_j for $0 \leq j \leq k$, with side conditions $s_{j-1} > s_j$ for $1 \leq j \leq k$. Using Lemma 21, add a scaling system forcing $s_0 = 2^E u$. For each $j = 1, \dots, k$, add the atom $u \mid s_{j-1} - s_j$. Since $s_{j-1} > s_j$ and $u > 0$, every solution determines a positive integer quotient

$$q_j = \frac{s_{j-1} - s_j}{u},$$

which encodes x_j . The choice of E ensures that every bounded solution of $\mathcal{I}$ can be encoded in this form.

For each affine form $L(\mathbf{x}) = c + \sum_{j=1}^k a_j x_j$ appearing in $\mathcal{I}$, define the homogeneous translated form $\widehat{L} = cu + \sum_{j=1}^k a_j (s_{j-1} - s_j)$. Thus, on every assignment satisfying the

atoms $u \mid s_{j-1} - s_j$, we have $\widehat{L} = uL(q_1, \dots, q_k)$. For every atom $L(\mathbf{x}) \mid M(\mathbf{x})$ of $\mathcal{I}$, add the translated atom $\widehat{L} \mid \widehat{M}$. The resulting system has size polynomial in $|\mathcal{I}|$: the only large number used as an exponent is E , whose binary length is polynomial, and Lemma 21 produces a system of size polynomial in $\log(E + 2)$.

We first check that the constructed system is merge-absorptive. By Lemma 40, the constraints enforcing $s_0 = 2^E u$ admit a sequence of merge atoms that puts s_0 , u , and all auxiliary variables used by those constraints into one component. Suppose that, after processing the first $j - 1$ atoms $u \mid s_{h-1} - s_h$, for $h = 1, \dots, j - 1$, the current component A contains u and s_{j-1} , while $B = \{s_j\}$ is still a singleton. The atom $u \mid s_{j-1} - s_j$ is a bounded-dividend merge atom: take $P_A = u$, $T_A = s_{j-1}$, and $R_B = -s_j$. The form u is positive on A , the form $-s_j$ is nonvanishing on B , and the side condition $s_{j-1} > s_j$ makes s_j side-condition bounded by A . Hence this atom merges s_j into A . Iterating this for $j = 1, \dots, k$ merges all variables introduced above. The translated atoms are then supported inside the single component, so they do not prevent all variables from being merged into one component.

It remains to prove equisatisfiability. Suppose first that $\mathcal{I}$ has a satisfying assignment $\xi_1, \dots, \xi_k$ with $\xi_j < 2^N$. Set $u = 1$, $s_0 = 2^E$, and define $s_j = s_{j-1} - \xi_j$ for $1 \leq j \leq k$. The choice of E gives $\xi_1 + \dots + \xi_k < 2^E$, so $s_0 > s_1 > \dots > s_k > 0$. The constraints enforcing $s_0 = 2^E u$ have a solution by Lemma 21, and every atom $u \mid s_{j-1} - s_j$ has quotient ξ_j for $j \leq k$. For each translated atom, $\widehat{L} = L(\xi_1, \dots, \xi_k)$ and $\widehat{M} = M(\xi_1, \dots, \xi_k)$, because $u = 1$. Therefore all translated divisibility atoms hold, and the constructed system is satisfiable.

Conversely, suppose the constructed system is satisfiable. For $j = 1, \dots, k$, define $q_j = (s_{j-1} - s_j)/u$. The atoms and the side conditions $s_{j-1} > s_j$ imply that every q_j is a positive integer. For every atom $L \mid M$ of $\mathcal{I}$, the translated atom gives $u L(q_1, \dots, q_k) \mid u M(q_1, \dots, q_k)$. Since $u > 0$, this is equivalent to $L(q_1, \dots, q_k) \mid M(q_1, \dots, q_k)$. Thus $q_1, \dots, q_k$ satisfies $\mathcal{I}$. This proves correctness of the polynomial-time many-one reduction. ◀

C Proofs for Succinct ALCs

The succinct model is defined in Remark 15. We first prove **succALC** = **FEXP**, and then give polynomial-size functional EPAD definitions of succinct ALC families, establishing the inclusion stated in Remark 26.

► **Proposition 51.** *The class of succinct ALC families with polynomial-time direct-connection algorithms is exactly **FEXP**: **succALC** = **FEXP**.*

Proof. From succinct ALC to FEXP. Let (C_n) be a succinct ALC family with index length $a(n) = \text{poly}(n)$ and word-width $W(n) \leq 2^{q(n)}$. Enumerate all gate indices in numerical order and evaluate their gates. An addition, a shift, or a schoolbook multiplication on $W(n)$ -bit words takes time polynomial in $W(n)$; the complete index table occupies at most $2^{a(n)} W(n) = 2^{\text{poly}(n)}$ bits. The direct-connection algorithm contributes only polynomial time per gate. Hence the circuit is evaluable in deterministic exponential time, proving **succALC** $\subseteq$ **FEXP**.

From FEXP to succinct ALC. Conversely, let a deterministic transducer run for $T(n) = 2^{p(n)}$ steps. The standard tableau construction gives a bounded-fan-in Boolean circuit of size $T(n)^{O(1)}$ for its computation: a gate is indexed by a time, tape position, track, and a constant-size local subgate. Its predecessors are computable from that index in polynomial time. Include $T(n)$ output cells, padded by leading zeros, and choose an ALC word-width of at least $T(n)$.

A Boolean AND gate is the product of its one-bit inputs. Boolean negation is computed by

$$\neg a = ((a +_W 1) \ll_W (W - 1)) \gg (W - 1), \quad a \in \{0, 1\}.$$

Thus every bounded-fan-in Boolean gate has a constant-size ALC simulation. Pack the output bits $b_0, \dots, b_{W-1}$ as $\sum_i (b_i \ll_W i)$ using a balanced addition tree. The summands have disjoint supports, so the sum is the required word. The resulting circuit still has exponential size and a polynomial-time direct-connection algorithm. Therefore $\mathbf{FEXP} \subseteq \mathbf{succALC}$. ◀

► **Remark 52 (Explicit exponential-size families).** The same proof shows that $\mathbf{FEXP}$ is also exactly the class computed by exponential-time uniform ALC families with $2^{\text{poly}(n)}$ gates and word-width $2^{\text{poly}(n)}$, where, on input 1^n , the uniformity machine outputs the whole circuit. We use the succinct formulation because Theorem 54 must produce a polynomial-size EPAD formula: a gate-by-gate translation of an explicitly exponential circuit would be exponentially large, whereas the direct-connection algorithm gives a polynomial-size description of all gates.

C.1 Functional EPAD Definitions of Succinct ALCs

We represent the evaluation of a succinct ALC by a succinct Boolean circuit, apply a Tseitin 3CNF encoding, and then use Lemma 35. The variables representing the external inputs and the circuit output retain the first indices. Applying Corollary 34 then gives an EPAD formula defining the graph of the function computed by the circuit. We allow free word inputs so that such a computation can be composed with exponentially wide integers already defined by EPAD formulas; Corollary 55 uses the special case with no free word inputs.

► **Lemma 53 (Succinct Exact-1-in-3 formula for circuit evaluation).** *Let (C_n) be a succinct ALC family with n Boolean inputs and polynomially many free word inputs, each of a uniformly specified length at most the circuit word-width. There is a uniformly succinct Exact-1-in-3 family whose first variables are, in order, all Boolean input bits, the bits of each free word from least significant to most significant, and the output bits from least significant to most significant. For every assignment to these variables, the Exact-1-in-3 formula is satisfiable exactly when the assigned output word equals the value computed by C_n on the assigned input bits and words.*

Proof. Boolean expansion. Expand each arithmetic gate of word-width W into a bounded-fan-in Boolean circuit of size $W^{O(1)}$. Use ripple-carry addition, a schoolbook multiplier formed from the W^2 partial products and binary additions while retaining only output positions $0, \dots, W - 1$, and fixed rewiring for logical shifts. A Boolean gate index consists of the ALC gate index, bit positions, and a constant number of internal coordinates, so its predecessors are computable in polynomial time. The expanded Boolean circuit still has $2^{\text{poly}(n)}$ gates.

Tseitin 3CNF. Introduce one propositional variable for each Boolean gate and clauses enforcing the value of that gate from its predecessors. For an AND gate $g = a \wedge b$, use

$$(\neg g \vee a), \quad (\neg g \vee b), \quad (g \vee \neg a \vee \neg b).$$

These clauses are satisfied exactly when the variable for g has the correct value. Use analogous constant-size clauses for NOT, constants, and wire copies, and pad clauses of width below three by repeated literals. Introduce copy variables for the external input and output bits and tie them to the corresponding circuit wires. If a word input has declared length below

W , constrain its higher circuit-input bits to zero. Number the input-copy and output-copy variables first, in the order stated in the lemma. Fixing the input-copy variables determines a unique output word, and the 3CNF has a satisfying extension exactly when the output-copy variables contain that word. The clause oracle computes the indexed local gate constraint in polynomial time.

Conversion to Exact-1-in-3. Apply Lemma 35. The conversion retains the indices of the input-copy and output-copy variables and preserves, for every assignment to those variables, the existence of a satisfying extension. Its local gadget also preserves polynomial-time clause access. This gives the required uniformly succinct Exact-1-in-3 family. ◀

► **Theorem 54** (EPAD representation of succinct ALCs). *Let (C_n) be a succinct ALC family with polynomially many free word inputs $z_1, \dots, z_t$, each of a uniformly specified length at most $W(n)$, in addition to its Boolean inputs. One can construct in polynomial time an EPAD formula*

$$\Phi_n(\mathbf{x}, \mathbf{z}, y)$$

that is functional in y and satisfies

$$\Phi_n(\mathbf{x}, \mathbf{z}, y) \iff y = C_n(\mathbf{x}, \mathbf{z})$$

whenever each word input lies in its declared range and $0 \leq y < 2^{W(n)}$.

Proof. Use Lemma 53. Let the declared lengths of the word inputs be $L_1, \dots, L_t$. Set

$$o_1 = n, \quad o_h = n + \sum_{j < h} L_j \quad (2 \leq h \leq t), \quad o_y = n + \sum_{j=1}^t L_j,$$

and let $L = o_y + W(n)$ be the number of Boolean variables whose values are encoded by the integer Z . In EPAD, introduce Z and impose

$$Z = \sum_{i=0}^{n-1} 2^i x_i + \sum_{h=1}^t 2^{o_h} z_h + 2^{o_y} y$$

using signed scaling and affine addition. The summands occupy disjoint bit intervals, so this sum is exactly their concatenation. Constrain $0 \leq x_i \leq 1$ for every Boolean input, and constrain every word input and y to its specified range. Apply Corollary 34 to this Z . The first L variables of a satisfying assignment are then exactly the bitwise concatenation of $\mathbf{x}, \mathbf{z}, y$. By Lemma 53, satisfiability is equivalent to $y = C_n(\mathbf{x}, \mathbf{z})$. Determinism of C_n gives functionality in y . ◀

► **Corollary 55.** *One has $\mathbf{FEXP} \subseteq \mathbf{FEPAD}$.*

Proof. Combine Proposition 51 and Theorem 54 with no free word inputs. ◀

D NEXP-Hardness of Nonerasing Word Equations with Length Constraints

► **Theorem 56** (EPAD to nonerasing word constraints). *EPAD formula satisfiability many-one reduces in polynomial time to nonerasing satisfiability for positive Boolean combinations of word equations and quantifier-free Presburger length constraints over the alphabet $\{a, b\}$.*

Proof. We use the terminology of Day and Konefal [4]. A word equation is an equality $\alpha = \beta$ over terminal letters and word variables. A solution maps each word variable to a nonempty word and fixes the terminals. A length constraint is a quantifier-free Presburger formula over word lengths. Positive Boolean combinations use only conjunction and disjunction at the outer level. The length constraints themselves may be arbitrary quantifier-free Presburger formulas.

We first put the EPAD input into the form used by the reduction. Replace every integer variable x by $x^+ - x^-$, where x^+ and x^- range over $\mathbb{N}$. This is equisatisfiable and keeps all terms affine. Push negations to atoms. For each divisibility literal $L \mid M$, with L, M affine integer terms, introduce nonnegative variables for $|L|$ and $|M|$. The absolute-value constraints are Presburger, and $L \mid M$ over $\mathbb{Z}$ is equivalent to the resulting nonnegative divisibility atom. Then use the standard elimination of negated divisibility literals for linear arithmetic with divisibility [14, 13]. After adding only polynomially many existential arithmetic variables, this gives an equisatisfiable positive Boolean combination of quantifier-free Presburger conditions and unnegated divisibility atoms over nonnegative arithmetic variables. We use the divisibility convention fixed after Definition 3.

We represent a nonnegative arithmetic value n by a nonempty word of length $n + 1$. For a word variable X , write $\text{val}(X) = |X| - 1$. Each arithmetic variable x receives a word variable X , and Presburger conditions are translated by replacing x with $\text{val}(X)$. This is still a quantifier-free Presburger length constraint. The subtraction in $\text{val}(X)$ is only notation; it can be removed by moving constants to the other side.

A nonempty word is *primitive* if it is not a proper power. We use the commutation theorem for words: if $XY = YX$, then there is a word P and integers $m, n \geq 0$ such that $X = P^m$ and $Y = P^n$. In particular, every word commuting with a primitive word X lies in X^* [17, Section 1.3].

It remains to translate unnegated divisibility atoms. First consider positive lengths. For word variables X, Y , define

$$\text{Div}_+(X, Y) := XY = YX \wedge aX = Zb \wedge Za = aZ,$$

where Z is a fresh auxiliary word variable. For nonempty word values, $\text{Div}_+(X, Y)$ enforces exactly the relation $|X| \mid |Y|$ on positive lengths. The equation $Za = aZ$ forces $Z = a^j$ with $j \geq 1$, and then $aX = Zb$ forces $X = a^{j-1}b$. This word is primitive, so $XY = YX$ forces Y to be a positive power of X . Conversely, if $x \mid y$ with $x, y > 0$, use $Z = a^x$, $X = a^{x-1}b$, and $Y = X^{y/x}$.

To handle zero, define the following formula for shifted lengths:

$$\text{Div}_0(X, Y) := |Y| = 1 \vee (|U| + 1 = |X| \wedge |V| + 1 = |Y| \wedge \text{Div}_+(U, V)),$$

with fresh auxiliary word variables U, V and the auxiliary variable used inside Div_+ . For nonempty X, Y , this formula satisfies $\text{Div}_0(X, Y)$ if and only if $\text{val}(X) \mid \text{val}(Y)$. If $|Y| = 1$, then $\text{val}(Y) = 0$, and every nonnegative value divides 0. In the second disjunct, U and V have lengths $\text{val}(X)$ and $\text{val}(Y)$, respectively, so Div_+ gives positive divisibility. Conversely, if $\text{val}(X) \mid \text{val}(Y)$ and $\text{val}(Y) = 0$, the first disjunct applies. If $\text{val}(Y) > 0$, then $\text{val}(X) > 0$, and the positive-length construction for Div_+ gives suitable U, V . Thus the shifted encoding has exactly the following behavior: a positive divisor uses the second disjunct and Div_+ ; the case $0 \mid 0$ uses the first disjunct; and $0 \mid m$ with $m > 0$ is rejected because neither disjunct can apply.

For each unnegated divisibility atom $\alpha = L \mid M$, introduce fresh word variables A_α, B_α , add the length constraints $\text{val}(A_\alpha) = L$ and $\text{val}(B_\alpha) = M$, and conjoin $\text{Div}_0(A_\alpha, B_\alpha)$. All auxiliary variables are interpreted existentially by the target satisfiability problem.

The construction is equisatisfiable. From an arithmetic satisfying assignment, choose nonempty words of the prescribed shifted lengths. The divisibility formulas are satisfied by the previous paragraphs. Conversely, a nonerasing solution assigns each arithmetic variable x the value $|X| - 1$. The translated Presburger constraints have the same truth values as their arithmetic originals, and each Div_0 formula enforces the corresponding divisibility atom. The initial handling of signs and negated divisibility was logically equivalent, so the recovered arithmetic assignment satisfies the original EPAD formula.

The construction is polynomial. Each transformation is local, each affine term is encoded by a linear-size length constraint, and each divisibility atom contributes only constantly many word-equation atoms and length constraints, apart from the size of the terms it mentions. The only terminal letters used are a and b . ◀

Proof of Corollary 2. Combine Theorems 1 and 56. ◀